

METODOLOGIAS CSS



PROF. DR. WESLEY DOS REIS BEZERRA

ANO 2026

Metodologias CSS

Prof. Wesley dos Reis Bezerra

wesley.bezerra@ifc.edu.br

Sumário

Introdução.....	8
BEM (Block-Element-Modifier).....	9
Metodologia BEM (Block Element Modifier).....	9
Introdução.....	9
Desenvolvimento.....	9
Conclusão.....	10
Referências Bibliográficas.....	10
Sintaxe da Metodologia BEM.....	11
Introdução.....	11
Desenvolvimento.....	11
Conclusão.....	15
Referências Bibliográficas.....	15
Exemplo Prático da Metodologia BEM.....	15
Introdução.....	15
Desenvolvimento.....	15
Conclusão.....	19
Referências Bibliográficas.....	19
Exercício Prático – Aplicando a Metodologia BEM.....	19
Introdução.....	19
Desenvolvimento.....	20
Conclusão.....	21
Referências Bibliográficas.....	22
Resolução do Exercício Prático – Metodologia BEM.....	22
Introdução.....	22
Desenvolvimento.....	22

Conclusão.....	28
Referências Bibliográficas.....	28
SMACSS.....	30
Metodologia SMACSS.....	30
Introdução.....	30
Desenvolvimento.....	30
Conclusão.....	32
Referências Bibliográficas.....	33
Sintaxe da Metodologia SMACSS.....	33
Introdução.....	33
Desenvolvimento.....	33
Conclusão.....	39
Referências Bibliográficas.....	40
Exemplo Prático da Metodologia SMACSS.....	40
Introdução.....	40
Desenvolvimento.....	40
Conclusão.....	45
Referências Bibliográficas.....	45
Exercício Prático – Aplicando a Metodologia SMACSS.....	46
Introdução.....	46
Desenvolvimento.....	46
Conclusão.....	49
Referências Bibliográficas.....	50
Resolução do Exercício Prático – Metodologia SMACSS.....	50
Introdução.....	50
Desenvolvimento.....	50
Conclusão.....	59

Referências Bibliográficas.....	59
OOCSS.....	60
Metodologia OOCSS.....	60
Introdução.....	60
Desenvolvimento.....	60
Conclusão.....	63
Referências Bibliográficas.....	63
Sintaxe da Metodologia OOCSS.....	64
Introdução.....	64
Desenvolvimento.....	64
Conclusão.....	69
Referências Bibliográficas.....	69
Exemplo Prático da Metodologia OOCSS.....	70
Introdução.....	70
Desenvolvimento.....	70
Conclusão.....	74
Referências Bibliográficas.....	74
Exercício Prático – Aplicando a Metodologia OOCSS.....	75
Introdução.....	75
Desenvolvimento.....	75
Conclusão.....	79
Referências Bibliográficas.....	79
Resolução do Exercício Prático – Metodologia OOCSS.....	79
Introdução.....	79
Desenvolvimento.....	80
Conclusão.....	89
Referências Bibliográficas.....	89

SUITCSS.....	90
Sintaxe.....	90
Exemplo de SUITCSS.....	90
Atividade de Fixação.....	90

Introdução

A utilização de metodologia para criação, manutenção, e distribuição do CSS é um fator muito importante para projeto com maior nível de profissionalismo. Mesmo que aprendamos a construir o CSS do nosso próprio jeito e este funcione bem, a manutenção futura deste tipo de código não planejado pode ser um problema.

É importante falar que não é ruim construir o código CSS deixando a sua inspiração fluir, o ruim é não voltar depois ao mesmo código e reorganizá-lo. Uma vez que nós programadores geralmente mantemos um grande numero de diferentes projetos ou ainda que temos que resgatar e dar manutenção em projetos antigos, a refatoração e organização do código do CSS é um fator muito importante para o sucesso na manutenção do seu código.

Outro aspecto importante de adotarmos uma metodologia de CSS é que planejaremos como os arquivos serão baixados. Um problema em projetos muito grandes é que os arquivos de folha de estilo tendem a tornar-se muito grandes e por vezes todo o código esta dentro de um mesmo arquivo. Isto faz com que uma pagina que exiba somente um formulário de contato, por exemplo, tenha que carregar o CSS de tabelas, imagens e etc; os quais não aparecerão nela. Este fato pode ser um problema para o desempenho da aplicação.

Nesta perspectivas temos alguma propostas de metodologia para manutenção de códigos CSS. Ainda que a lista não esgote o assunto, ela traz as metodologias mais utilizadas atualmente e apresentam conceitos gerais sobre cada metodologia. Assim você poderá escolher qual a metodologia que mais se alinha com a sua forma de trabalho, ou ainda, a que se alinha com a forma de trabalho que você deseja adotar.

Vejamos agora uma lista de metodologias:

- BEM (Block-Element-Modifier)
- SMACSS
- OOCSS
- SUITCSS

BEM (Block-Element-Modifier)

Metodologia BEM (Block Element Modifier)

Introdução

BEM é uma metodologia de organização de código CSS criada pela empresa russa Yandex com o objetivo de tornar o desenvolvimento de interfaces mais previsível, escalável e de fácil manutenção. O nome BEM é um acrônimo para *Block, Element, Modifier* (Bloco, Elemento e Modificador), três conceitos fundamentais que definem a estrutura dos componentes de uma interface.

A principal proposta da metodologia é reduzir problemas comuns em projetos CSS, como conflitos de seletores, excesso de especificidade e dificuldade de reutilização de componentes. Por meio de uma convenção padronizada de nomenclatura, o desenvolvedor consegue identificar rapidamente a função de cada classe e sua relação com os demais elementos da página.

Desenvolvimento

A metodologia BEM é baseada no conceito de componentes independentes. Cada componente da interface é considerado um **bloco**, que representa uma unidade funcional autônoma. Um bloco pode ser um menu, um formulário, um cartão de produto, um cabeçalho ou qualquer outro componente reutilizável da aplicação. A nomenclatura de um bloco normalmente utiliza apenas seu nome, por exemplo:

```
.menu {}  
.card {}  
.formulario {}
```

Dentro de um bloco podem existir **elementos**, que representam partes internas do componente e dependem diretamente dele para fazer sentido. Na convenção BEM, os elementos são identificados pelo uso de dois sublinhados (__) após o nome do bloco.

```
.menu__item {}  
.menu__link {}  
.card__titulo {}  
.card__descricao {}
```

Além dos blocos e elementos, a metodologia utiliza os **modificadores**, responsáveis por representar variações visuais ou comportamentais de um componente. Os modificadores são indicados por dois hífens (- -) após o nome do bloco ou elemento.

```
.botao--primario {}  
.botao--secundario {}
```

```
.card--destacado {}  
.menu__item--ativo {}
```

Uma das características mais importantes do BEM é a independência dos componentes. Em vez de utilizar seletores fortemente acoplados à estrutura do HTML, como:

```
.menu ul li a {  
    color: blue;  
}
```

a metodologia recomenda o uso de classes específicas:

```
.menu__link {  
    color: blue;  
}
```

Essa abordagem reduz a dependência da estrutura do documento e facilita alterações futuras sem causar efeitos colaterais em outras partes do sistema.

Em projetos de grande porte, a adoção do BEM favorece a modularização do código, melhora a colaboração entre equipes e simplifica a manutenção. Como cada classe possui um significado explícito, novos desenvolvedores conseguem compreender rapidamente a organização dos componentes e suas respectivas responsabilidades.

Conclusão

A metodologia BEM é uma das convenções mais populares para organização de CSS devido à sua simplicidade e eficiência. Sua estrutura baseada em blocos, elementos e modificadores promove a criação de componentes reutilizáveis, reduz conflitos entre estilos e melhora significativamente a legibilidade do código.

Embora a nomenclatura possa resultar em nomes de classes mais extensos, esse pequeno aumento na quantidade de caracteres é compensado pela facilidade de manutenção, escalabilidade e previsibilidade do projeto. Por esse motivo, o BEM continua sendo amplamente utilizado em aplicações web modernas e em projetos que exigem organização consistente do código CSS.

Referências Bibliográficas

GETBEM. *BEM — Block Element Modifier*. Disponível em: <https://getbem.com>. Acesso em: 25 maio 2026.

FRANKLIN, Ben. *CSS Architecture for Modern Web Applications*. Birmingham: Packt Publishing, 2016.

SMACSS. *Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

VEROU, Lea. *CSS Secrets: Better Solutions to Everyday Web Design Problems*. Sebastopol: O'Reilly Media, 2015.

YANDEX. *BEM Methodology Documentation*. Disponível em: <https://en.bem.info>. Acesso em: 25 maio 2026.

Sintaxe da Metodologia BEM

Introdução

A metodologia BEM utiliza uma convenção de nomenclatura específica para definir componentes de interface e seus estados. Essa convenção foi criada para tornar o código CSS mais legível, previsível e escalável, permitindo que desenvolvedores identifiquem rapidamente a função de uma classe apenas pelo seu nome.

A sintaxe do BEM é baseada em três elementos fundamentais: **Block (Bloco)**, **Element (Elemento)** e **Modifier (Modificador)**. Cada um possui uma forma específica de escrita e uma finalidade bem definida dentro da estrutura do componente.

Desenvolvimento

Bloco (Block)

O bloco representa um componente independente da interface. Ele deve possuir significado próprio e não depender de outros elementos para existir.

Sintaxe:

```
.bloco {}
```

Exemplos:

```
.menu {}  
.card {}  
.formulario {}  
.cabecalho {}  
.rodape {}
```

Os nomes dos blocos geralmente são escritos em letras minúsculas e palavras compostas são separadas por hífen.

```
.lista-produtos {}  
.menu-principal {}  
.painel-controle {}
```

Elemento (Element)

O elemento representa uma parte interna de um bloco. Ele não possui significado isoladamente e depende do bloco para existir.

Sintaxe:

```
.bloco__elemento {}
```

A separação entre bloco e elemento é realizada utilizando dois sublinhados (__).

Exemplos:

```
.menu__item {}
```

```
.menu__link {}
```

```
.card__titulo {}
```

```
.card__imagem {}
```

```
.card__descricao {}
```

```
.formulario__campo {}
```

```
.formulario__botao {}
```

Um elemento sempre pertence a um bloco específico.

```
.card__titulo {}
```

```
.card__conteudo {}
```

```
.card__rodape {}
```

Modificador (Modifier)

O modificador representa uma variação de aparência, comportamento ou estado de um bloco ou elemento.

Sintaxe para bloco:

```
.bloco--modificador {}
```

Sintaxe para elemento:

```
.bloco__elemento--modificador {}
```

A separação entre o componente e sua variação é realizada por meio de dois hífen (- -).

Exemplos:

```
.botao--primario {}
```

```
.botao--secundario {}
```

```
.card--destacado {}  
.card--horizontal {}  
  
.menu__item--ativo {}  
.menu__link--desabilitado {}
```

Estrutura Completa

Um componente BEM geralmente combina os três conceitos.

HTML:

```
<div class="card card--destacado">  
    
  
  <h2 class="card__titulo">  
    Notebook Gamer  
  </h2>  
  
  <p class="card__descricao">  
    Equipamento voltado para aplicações de alto desempenho.  
  </p>  
  
  <button class="card__botao card__botao--comprar">  
    Comprar  
  </button>  
</div>
```

CSS:

```
.card {  
  padding: 20px;  
  border: 1px solid #ccc;  
}  
  
.card--destacado {  
  border-color: #0077cc;  
}  
  
.card__imagem {  
  width: 100%;  
}
```

```
.card__titulo {
  font-size: 1.5rem;
}

.card__descricao {
  color: #555;
}

.card__botao {
  padding: 10px 20px;
}

.card__botao--comprar {
  background-color: #0077cc;
  color: white;
}
```

Regras Gerais de Nomenclatura

Durante a utilização do BEM, algumas convenções são recomendadas:

- Utilizar apenas classes para estilização.
- Evitar seletores baseados em tags HTML.
- Evitar aninhamentos excessivos.
- Utilizar hífen para separar palavras compostas.
- Utilizar `__` para elementos.
- Utilizar `--` para modificadores.
- Criar nomes descritivos e relacionados à função do componente.

Exemplos válidos:

```
.menu-principal {}
.menu-principal__item {}
.menu-principal__item--ativo {}

.lista-produtos {}
.lista-produtos__produto {}
.lista-produtos__produto--promocao {}
```

Exemplos não recomendados:

```
.menu ul li a {}  
.card .titulo {}  
#cabecalho .menu .item {}
```

Conclusão

A sintaxe do BEM foi projetada para representar explicitamente a estrutura e o propósito de cada componente da interface. A utilização consistente de blocos, elementos e modificadores permite a criação de códigos CSS mais organizados, reutilizáveis e fáceis de manter.

Embora os nomes das classes possam se tornar mais extensos quando comparados a outras abordagens, a clareza proporcionada pela convenção reduz ambiguidades e facilita o trabalho colaborativo em projetos de médio e grande porte.

Referências Bibliográficas

GETBEM. *BEM — Block Element Modifier*. Disponível em: <https://getbem.com>. Acesso em: 25 maio 2026.

YANDEX. *BEM Methodology Documentation*. Disponível em: <https://en.bem.info>. Acesso em: 25 maio 2026.

COYIER, Chris. *BEM 101*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *Scalable and Modular Architecture for CSS (SMACSS)*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

VEROU, Lea. *CSS Secrets: Better Solutions to Everyday Web Design Problems*. Sebastopol: O'Reilly Media, 2015.

Exemplo Prático da Metodologia BEM

Introdução

Uma das melhores formas de compreender a metodologia BEM é por meio da construção de um componente real de interface. Neste exemplo será desenvolvido um cartão de produto (*product card*), componente amplamente utilizado em sistemas de comércio eletrônico, catálogos digitais e marketplaces.

O objetivo é demonstrar como os conceitos de **Block**, **Element** e **Modifier** são aplicados na prática para criar uma estrutura organizada, reutilizável e de fácil manutenção.

Desenvolvimento

O componente principal será o cartão do produto. Como ele possui significado próprio e pode ser reutilizado em diferentes partes do sistema, será definido como um **bloco**.

Estrutura HTML

```
<div class="produto produto--destaque">

    

    <div class="produto__conteudo">

        <h2 class="produto__titulo">
            Notebook Gamer RTX
        </h2>

        <p class="produto__descricao">
            Equipamento de alto desempenho destinado ao desenvolvimento,
            modelagem 3D e execução de aplicações avançadas.
        </p>

        <span class="produto__preco">
            R$ 6.499,00
        </span>

        <button class="produto__botao produto__botao--comprar">
            Comprar
        </button>

    </div>

</div>
```

Neste exemplo:

- **produto** é o bloco.
- **produto__imagem**, **produto__conteudo**, **produto__titulo**, **produto__descricao**, **produto__preco** e **produto__botao** são elementos.
- **produto--destaque** e **produto__botao--comprar** são modificadores.

Estrutura CSS

```
.produto {  
    display: flex;  
    gap: 20px;  
    padding: 20px;  
    border: 1px solid #dcdcdc;  
    border-radius: 8px;  
}
```

```
.produto--destaque {  
    border: 2px solid #0066cc;  
    background-color: #f5f9ff;  
}
```

```
.produto__imagem {  
    width: 250px;  
    height: auto;  
}
```

```
.produto__conteudo {  
    display: flex;  
    flex-direction: column;  
}
```

```
.produto__titulo {  
    font-size: 1.8rem;  
    margin-bottom: 10px;  
}
```

```
.produto__descricao {  
    margin-bottom: 15px;  
    line-height: 1.6;  
}
```

```
.produto__preco {  
    font-size: 1.4rem;  
    font-weight: bold;  
    margin-bottom: 15px;
```

```
}
```

```
.produto__botao {  
  padding: 12px 24px;  
  border: none;  
  cursor: pointer;  
}
```

```
.produto__botao--comprar {  
  background-color: #0066cc;  
  color: white;  
}
```

Observe que cada classe descreve claramente sua função dentro do componente. Não há necessidade de utilizar seletores complexos como:

```
.produto div h2 {}  
.produto div button {}  
.produto img {}
```

A responsabilidade de cada elemento fica explícita por meio da nomenclatura BEM.

Variações do Componente

Uma das maiores vantagens do BEM é a facilidade para criar versões diferentes do mesmo componente utilizando modificadores.

Por exemplo:

```
<div class="produto produto--promocao">  
  ...  
</div>
```

```
.produto--promocao {  
  background-color: #fff8e6;  
  border-color: #ff9900;  
}
```

Outra possibilidade é criar estados para o botão:

```
<button  
  class="produto__botao produto__botao--desabilitado">  
  Indisponível  
</button>
```

```
.produto__botao--desabilitado {  
    background-color: #999;  
    cursor: not-allowed;  
}
```

Perceba que não foi necessário alterar a estrutura original do componente. Apenas novos modificadores foram adicionados para representar diferentes estados e comportamentos.

Conclusão

A metodologia BEM permite modelar componentes de interface de forma clara e organizada. No exemplo apresentado, o cartão de produto foi dividido em um bloco principal, elementos internos e modificadores responsáveis por representar variações visuais e estados específicos.

Essa abordagem facilita a reutilização dos componentes, reduz conflitos entre estilos e melhora significativamente a manutenção do código, especialmente em aplicações corporativas, sistemas web de grande porte e projetos desenvolvidos por equipes multidisciplinares.

Referências Bibliográficas

GETBEM. *BEM — Block Element Modifier*. Disponível em: <https://getbem.com>. Acesso em: 25 maio 2026.

YANDEX. *BEM Methodology Documentation*. Disponível em: <https://en.bem.info>. Acesso em: 25 maio 2026.

COYIER, Chris. *BEM 101*. Disponível em: <https://css-tricks.com/bem-101/>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *Scalable and Modular Architecture for CSS (SMACSS)*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

VEROU, Lea. *CSS Secrets: Better Solutions to Everyday Web Design Problems*. Sebastopol: O'Reilly Media, 2015.

Exercício Prático – Aplicando a Metodologia BEM

Introdução

A aplicação da metodologia BEM exige a capacidade de identificar componentes independentes, suas partes internas e possíveis variações de aparência ou comportamento. O domínio dessa técnica é fundamental para o desenvolvimento de interfaces escaláveis e de fácil manutenção.

Neste exercício, o estudante deverá analisar um cenário realista de desenvolvimento web e estruturar corretamente as classes CSS utilizando a convenção BEM.

Desenvolvimento

Situação-Problema

Uma empresa deseja desenvolver a página inicial de seu portal corporativo. Nessa página existe um componente denominado **Notícia**, responsável por exibir informações institucionais.

Cada notícia deve conter:

- Imagem de destaque;
- Título;
- Resumo;
- Data de publicação;
- Botão "Ler mais".

Além disso, existem duas variações desse componente:

- Notícia em destaque;
- Notícia arquivada.

O botão também pode apresentar dois estados:

- Normal;
- Desabilitado.

Atividades

Questão 1

Identifique o **bloco principal** do componente e justifique sua escolha.

Questão 2

Liste todos os **elementos** pertencentes ao bloco principal utilizando a nomenclatura BEM.

Questão 3

Defina os **modificadores** necessários para representar as variações do componente e dos seus elementos.

Questão 4

Escreva a estrutura HTML do componente utilizando exclusivamente classes BEM.

Questão 5

Crie um arquivo CSS contendo as definições das seguintes classes:

- Bloco principal;
- Elementos internos;
- Modificador de notícia em destaque;
- Modificador de notícia arquivada;
- Modificador de botão desabilitado.

Utilize propriedades CSS adequadas para diferenciar visualmente cada estado.

Questão 6

Explique quais vantagens a metodologia BEM oferece em relação à seguinte abordagem:

```
.noticias div h2 {}  
.noticias div p {}  
.noticias button {}
```

Sua resposta deve abordar:

- Legibilidade;
- Reutilização;
- Manutenção;
- Escalabilidade.

Desafio Adicional

Considere que o portal agora possui também os componentes:

- Evento;
- Comunicado;
- Vaga de Emprego.

Modele a nomenclatura BEM para cada um desses componentes, definindo:

- O bloco;
- Pelo menos três elementos;
- Pelo menos dois modificadores.

Conclusão

Este exercício permite praticar os principais conceitos da metodologia BEM em um contexto semelhante ao encontrado em projetos profissionais. A correta identificação de blocos, elementos e

modificadores contribui para a construção de folhas de estilo mais organizadas, reduzindo dependências estruturais e facilitando a evolução da interface ao longo do tempo.

Ao final da atividade, o estudante deverá ser capaz de modelar componentes reutilizáveis utilizando uma convenção de nomenclatura padronizada e amplamente adotada no desenvolvimento front-end moderno.

Referências Bibliográficas

GETBEM. *BEM — Block Element Modifier*. Disponível em: <https://getbem.com>. Acesso em: 25 maio 2026.

YANDEX. *BEM Methodology Documentation*. Disponível em: <https://en.bem.info>. Acesso em: 25 maio 2026.

COYIER, Chris. *BEM 101*. Disponível em: <https://css-tricks.com/bem-101/>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

Resolução do Exercício Prático – Metodologia BEM

Introdução

Nesta resolução será aplicada a metodologia BEM para modelar um componente de notícias de um portal corporativo. O objetivo é demonstrar como identificar corretamente blocos, elementos e modificadores, além de evidenciar as vantagens dessa abordagem na organização do código CSS.

A solução apresentada segue as boas práticas recomendadas pela metodologia, priorizando a clareza semântica e a reutilização dos componentes.

Desenvolvimento

Questão 1 – Identificação do Bloco

O bloco principal será:

```
.noticia
```

Justificativa:

O componente "Notícia" possui significado próprio e pode ser utilizado de forma independente em diferentes partes do sistema, como página inicial, área administrativa, resultados de busca e páginas de categoria. Portanto, ele representa um bloco na metodologia BEM.

Questão 2 – Identificação dos Elementos

Os elementos internos do bloco são:

```
.noticia__imagem  
.noticia__titulo  
.noticia__resumo  
.noticia__data  
.noticia__botao
```

Cada elemento depende diretamente do bloco `noticia` para fazer sentido dentro da interface.

Questão 3 – Definição dos Modificadores

Modificadores do bloco

```
.noticia--destaque  
.noticia--arquivada
```

Modificadores do botão

```
.noticia__botao--desabilitado
```

Esses modificadores representam variações visuais e comportamentais do componente.

Questão 4 – Estrutura HTML

```
<article class="noticia noticia--destaque">
```

```
  
```

```
  <h2 class="noticia__titulo">  
    Universidade inaugura novo laboratório  
  </h2>
```

```
  <p class="noticia__resumo">
```

```
    O novo espaço foi desenvolvido para apoiar
    pesquisas nas áreas de inteligência artificial
    e ciência de dados.
</p>
```

```
<span class="noticia__data">
    25/05/2026
</span>
```

```
<button class="noticia__botao">
    Ler mais
</button>
```

```
</article>
```

Exemplo com notícia arquivada:

```
<article class="noticia noticia--arquivada">
```

```
    
```

```
    <h2 class="noticia__titulo">
        Evento Encerrado
    </h2>
```

```
    <p class="noticia__resumo">
        Esta notícia foi arquivada.
    </p>
```

```
    <span class="noticia__data">
        10/03/2025
    </span>
```

```
    <button
        class="noticia__botao noticia__botao--desabilitado">
        Indisponível
    </button>
```

</article>

Questão 5 – Implementação CSS

```
.noticia {  
    width: 400px;  
    padding: 20px;  
    border: 1px solid #dcdcdc;  
    border-radius: 8px;  
    font-family: Arial, sans-serif;  
}
```

```
.noticia__imagem {  
    width: 100%;  
    margin-bottom: 15px;  
}
```

```
.noticia__titulo {  
    font-size: 1.5rem;  
    margin-bottom: 10px;  
}
```

```
.noticia__resumo {  
    line-height: 1.6;  
    margin-bottom: 10px;  
}
```

```
.noticia__data {  
    display: block;  
    margin-bottom: 15px;  
    color: #666;  
}
```

```
.noticia__botao {  
    padding: 10px 20px;  
    border: none;  
    background-color: #0055aa;
```

```
    color: white;
    cursor: pointer;
}

.noticia--destaque {
    border: 2px solid #0055aa;
    background-color: #eef5ff;
}

.noticia--arquivada {
    opacity: 0.7;
    background-color: #f4f4f4;
}

.noticia__botao--desabilitado {
    background-color: #999;
    cursor: not-allowed;
}
```

Questão 6 – Comparação com Seletores Tradicionais

Considere a seguinte estrutura:

```
.noticias div h2 {}
.noticias div p {}
.noticias button {}
```

Legibilidade

Na abordagem tradicional, a finalidade dos seletores não é imediatamente identificável. Já no BEM, nomes como:

```
.noticia__titulo
.noticia__resumo
.noticia__botao
```

descrevem claramente o papel de cada elemento.

Reutilização

Classes BEM podem ser reutilizadas em diferentes contextos sem depender da estrutura HTML. Isso facilita a criação de componentes independentes.

Manutenção

Alterações na estrutura do HTML tendem a impactar menos o CSS quando se utiliza BEM, pois os estilos estão associados às classes e não à hierarquia dos elementos.

Escalabilidade

Projetos grandes podem conter centenas de componentes. A padronização fornecida pelo BEM reduz conflitos de nomenclatura e facilita o trabalho colaborativo entre equipes de desenvolvimento.

Desafio Adicional

Componente Evento

```
.evento
.evento__titulo
.evento__descricao
.evento__data

.evento--online
.evento--presencial
```

Componente Comunicado

```
.comunicado
.comunicado__titulo
.comunicado__texto
.comunicado__autor

.comunicado--urgente
.comunicado--interno
```

Componente Vaga

```
.vaga
.vaga__cargo
.vaga__descricao
.vaga__salario

.vaga--aberta
.vaga--encerrada
```

Exemplo:

```
<div class="vaga vaga--aberta">

    <h2 class="vaga__cargo">
        Desenvolvedor Front-End
    </h2>

    <p class="vaga__descricao">
        Desenvolvimento de aplicações web corporativas.
    </p>

    <span class="vaga__salario">
        R$ 8.000,00
    </span>

</div>
```

Conclusão

A resolução demonstra como a metodologia BEM organiza componentes de interface por meio de uma nomenclatura consistente baseada em blocos, elementos e modificadores. Essa abordagem favorece a modularização do código, reduz a dependência da estrutura HTML e melhora a manutenção de projetos de médio e grande porte.

Ao dominar essa metodologia, o desenvolvedor passa a construir componentes mais previsíveis, reutilizáveis e escaláveis, características essenciais para aplicações web modernas.

Referências Bibliográficas

GETBEM. *BEM — Block Element Modifier*. Disponível em: <https://getbem.com>. Acesso em: 25 maio 2026.

YANDEX. *BEM Methodology Documentation*. Disponível em: <https://en.bem.info>. Acesso em: 25 maio 2026.

COYIER, Chris. *BEM 101*. Disponível em: <https://css-tricks.com/bem-101/>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *Scalable and Modular Architecture for CSS (SMACSS)*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

SMACSS

Metodologia SMACSS

Introdução

SMACSS (*Scalable and Modular Architecture for CSS*) é uma metodologia de organização de código CSS criada por Jonathan Snook com o objetivo de tornar as folhas de estilo mais escaláveis, modulares e fáceis de manter. Diferentemente de metodologias que focam principalmente em convenções de nomenclatura, como o BEM, o SMACSS propõe uma arquitetura para estruturar e categorizar os estilos de um projeto.

A principal ideia do SMACSS é dividir o CSS em grupos de responsabilidades bem definidas. Essa separação facilita a compreensão do código, reduz a duplicação de regras e permite que equipes trabalhem em projetos complexos de forma mais organizada.

Desenvolvimento

O SMACSS classifica os estilos em cinco categorias principais: **Base**, **Layout**, **Module**, **State** e **Theme**. Cada categoria possui uma função específica dentro da arquitetura da aplicação.

Base

A categoria Base contém os estilos fundamentais aplicados aos elementos HTML sem a necessidade de classes específicas. São regras responsáveis por definir comportamentos padrão para elementos como títulos, parágrafos, listas, formulários e links.

Exemplo:

```
body {  
    font-family: Arial, sans-serif;  
    line-height: 1.6;  
}  
  
h1 {  
    font-size: 2rem;  
}  
  
a {  
    text-decoration: none;  
}
```

Esses estilos estabelecem uma base consistente para toda a aplicação.

Layout

Os estilos de Layout definem a estrutura geral da página, organizando regiões como cabeçalho, menu lateral, conteúdo principal e rodapé.

Exemplo:

```
.l-header {  
    height: 80px;  
}
```

```
.l-sidebar {  
    width: 250px;  
}
```

```
.l-content {  
    flex: 1;  
}
```

É comum utilizar o prefixo `l-` para identificar elementos relacionados ao layout.

Module

Os módulos representam componentes independentes e reutilizáveis da interface. Um módulo pode ser um menu, um cartão de produto, um formulário, uma tabela ou qualquer outro componente reutilizável.

Exemplo:

```
.card {  
    border: 1px solid #ccc;  
    padding: 20px;  
}
```

```
.card-title {  
    font-size: 1.5rem;  
}
```

Os módulos constituem a maior parte do CSS em aplicações modernas.

State

A categoria State define estados temporários ou condições específicas de um componente.

Exemplos:

```
.is-active {
```

```
    display: block;
}
```

```
.is-hidden {
    display: none;
}
```

```
.is-disabled {
    opacity: 0.5;
}
```

Os prefixos `is-` e `has-` são amplamente utilizados para indicar estados.

Theme

A categoria Theme é responsável pela personalização visual da aplicação, permitindo alterar cores, tipografia e outros aspectos estéticos sem modificar a estrutura dos componentes.

Exemplo:

```
.theme-dark {
    background-color: #222;
    color: #fff;
}
```

```
.theme-light {
    background-color: #fff;
    color: #222;
}
```

Essa abordagem facilita a implementação de múltiplos temas visuais em uma mesma aplicação.

Conclusão

A metodologia SMACSS fornece uma arquitetura organizada para projetos CSS ao separar os estilos de acordo com suas responsabilidades. Essa divisão melhora a legibilidade do código, reduz a complexidade de manutenção e favorece a escalabilidade de aplicações web.

Embora não imponha regras rígidas de nomenclatura, como ocorre no BEM, o SMACSS oferece uma estrutura conceitual sólida para organizar folhas de estilo em projetos de médio e grande porte. Por essa razão, continua sendo uma das arquiteturas CSS mais influentes no desenvolvimento front-end moderno.

Referências Bibliográficas

SNOOK, Jonathan. *Scalable and Modular Architecture for CSS (SMACSS)*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com/book>. Acesso em: 25 maio 2026.

COYIER, Chris. *Comparing CSS Methodologies*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

VEROU, Lea. *CSS Secrets: Better Solutions to Everyday Web Design Problems*. Sebastopol: O'Reilly Media, 2015.

Sintaxe da Metodologia SMACSS

Introdução

Diferentemente de metodologias como BEM, que possuem uma convenção de nomenclatura rígida para a criação de classes, o SMACSS não define uma sintaxe obrigatória. Seu foco está na organização arquitetural do CSS por meio da categorização dos estilos de acordo com suas responsabilidades dentro da aplicação.

Apesar de não impor regras estritas, o SMACSS sugere padrões de nomenclatura que auxiliam na identificação da finalidade de cada classe. Esses padrões permitem que os desenvolvedores reconheçam rapidamente se um estilo pertence ao layout, a um módulo, a um estado ou a um tema.

Desenvolvimento

Sintaxe para Base

A categoria Base normalmente utiliza seletores HTML puros, sem a necessidade de classes.

Sintaxe:

```
elemento {  
    propriedade: valor;  
}
```

Exemplos:

```
body {  
    font-family: Arial, sans-serif;  
}
```

```
h1 {  
    font-size: 2rem;  
}  
  
p {  
    line-height: 1.6;  
}  
  
a {  
    text-decoration: none;  
}
```

Os estilos Base estabelecem os padrões globais da aplicação.

Sintaxe para Layout

Os componentes de layout geralmente recebem o prefixo `l` - (*layout*).

Sintaxe:

```
.l-nome-layout {  
    propriedade: valor;  
}
```

Exemplos:

```
.l-header {  
    height: 80px;  
}
```

```
.l-sidebar {  
    width: 250px;  
}
```

```
.l-content {  
    flex: 1;  
}
```

```
.l-footer {  
    padding: 20px;  
}
```

Essa convenção facilita a identificação das áreas estruturais da página.

Sintaxe para Module

Os módulos representam componentes independentes e reutilizáveis.

Sintaxe:

```
.modulo {  
    propriedade: valor;  
}
```

ou

```
.modulo-subcomponente {  
    propriedade: valor;  
}
```

Exemplos:

```
.card {  
    border: 1px solid #ccc;  
}
```

```
.card-title {  
    font-size: 1.5rem;  
}
```

```
.card-description {  
    color: #555;  
}
```

Outro exemplo:

```
.menu {}  
.menu-item {}  
.menu-link {}
```

O SMACSS não exige delimitadores específicos como `__` ou `--`, ficando a critério da equipe estabelecer padrões internos.

Sintaxe para State

Os estados normalmente utilizam os prefixos `is-` ou `has-`.

Sintaxe:

```
.is-estado {  
    propriedade: valor;
```

```
}
```

ou

```
.has-caracteristica {  
    propriedade: valor;  
}
```

Exemplos:

```
.is-active {  
    display: block;  
}
```

```
.is-hidden {  
    display: none;  
}
```

```
.is-disabled {  
    opacity: 0.5;  
}
```

```
.has-error {  
    border: 1px solid red;  
}
```

Aplicação em HTML:

```
<button class="botao is-disabled">  
    Salvar  
</button>
```

ou

```
<li class="menu-item is-active">  
    Início  
</li>
```

Sintaxe para Theme

Os temas geralmente recebem o prefixo `theme-`.

Sintaxe:

```
.theme-nome {
```

```
    propriedade: valor;
}
```

Exemplos:

```
.theme-dark {
    background-color: #222;
    color: white;
}
```

```
.theme-light {
    background-color: white;
    color: #222;
}
```

Aplicação:

```
<body class="theme-dark">
```

Estrutura Completa Utilizando SMACSS

HTML

```
<header class="l-header">
```

```
    <nav class="menu">
```

```
        <ul>
```

```
            <li class="menu-item is-active">
```

```
                <a class="menu-link" href="#">
```

```
                    Início
```

```
                </a>
```

```
            </li>
```

```
            <li class="menu-item">
```

```
                <a class="menu-link" href="#">
```

```
                    Produtos
```

```
                </a>
```

```
            </li>
```

```
        </ul>
```

```
</nav>
```

```
</header>
```

```
<main class="l-content">
```

```
<article class="card">
```

```
<h2 class="card-title">
```

```
    Notebook Gamer
```

```
</h2>
```

```
<p class="card-description">
```

```
    Equipamento destinado a aplicações de alto desempenho.
```

```
</p>
```

```
<button class="button">
```

```
    Comprar
```

```
</button>
```

```
</article>
```

```
</main>
```

CSS

```
body {
```

```
    font-family: Arial, sans-serif;
```

```
}
```

```
.l-header {
```

```
    background-color: #333;
```

```
}
```

```
.l-content {
```

```
    padding: 20px;
```

```
}
```

```
.menu {
```

```
    display: flex;
```

```
}

.menu-item {
  list-style: none;
}

.menu-link {
  color: white;
}

.card {
  border: 1px solid #ccc;
  padding: 20px;
}

.card-title {
  font-size: 1.5rem;
}

.card-description {
  color: #666;
}

.is-active {
  font-weight: bold;
}
```

Nesse exemplo é possível identificar claramente as categorias propostas pelo SMACSS:

- `body` → Base
- `l-header` e `l-content` → Layout
- `menu`, `card` e `button` → Module
- `is-active` → State
- `theme-dark` (caso existisse) → Theme

Conclusão

A sintaxe do SMACSS é baseada em convenções e não em regras obrigatórias. Sua principal característica é a organização dos estilos em categorias funcionais, permitindo que o CSS seja estruturado de maneira mais lógica e escalável.

Ao utilizar prefixos como `l-`, `is-`, `has-` e `theme-`, os desenvolvedores conseguem identificar rapidamente a responsabilidade de cada classe, tornando o código mais legível e facilitando a manutenção de aplicações complexas.

Referências Bibliográficas

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS Book*. Disponível em: <https://smacss.com/book>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

COYIER, Chris. *Comparing CSS Methodologies*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

VEROU, Lea. *CSS Secrets: Better Solutions to Everyday Web Design Problems*. Sebastopol: O'Reilly Media, 2015.

Exemplo Prático da Metodologia SMACSS

Introdução

Para compreender a aplicação da metodologia SMACSS, é importante observar como os estilos são organizados em categorias de responsabilidade. Diferentemente do BEM, que enfatiza uma convenção de nomenclatura para componentes, o SMACSS concentra-se na arquitetura do CSS, separando regras de acordo com sua função dentro da aplicação.

Neste exemplo, será desenvolvido um sistema de notícias para um portal institucional. O objetivo é demonstrar a utilização das categorias **Base**, **Layout**, **Module**, **State** e **Theme** em um cenário próximo ao encontrado em aplicações corporativas.

Desenvolvimento

Estrutura HTML

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Portal Institucional</title>
  <link rel="stylesheet" href="style.css">
</head>
```

```
<body class="theme-light">
```

```
  <header class="l-header">
```

```
    <h1>Portal Institucional</h1>
```

```
  </header>
```

```
  <main class="l-content">
```

```
    <article class="news-card">
```

```
      
```

```
      <h2 class="news-title">
```

```
        Universidade inaugura novo laboratório
```

```
      </h2>
```

```
      <p class="news-description">
```

```
        O espaço foi projetado para apoiar pesquisas
```

```
        em inteligência artificial e ciência de dados.
```

```
      </p>
```

```
      <button class="button">
```

```
        Ler mais
```

```
      </button>
```

```
    </article>
```

```
    <article class="news-card is-featured">
```

```
      
```

```
      <h2 class="news-title">
```

```
        Congresso Internacional de Tecnologia
```

```
</h2>
```

```
<p class="news-description">
```

```
    Evento reunirá pesquisadores e profissionais  
    da área de computação.
```

```
</p>
```

```
<button class="button">
```

```
    Ler mais
```

```
</button>
```

```
</article>
```

```
</main>
```

```
<footer class="l-footer">
```

```
    Portal Institucional © 2026
```

```
</footer>
```

```
</body>
```

```
</html>
```

Categoria Base

Os estilos Base definem a aparência padrão dos elementos HTML.

```
body {  
    margin: 0;  
    font-family: Arial, sans-serif;  
    line-height: 1.6;  
}
```

```
h1,  
h2 {  
    margin: 0;  
}
```

```
img {  
    max-width: 100%;  
}
```

```
button {  
    font-size: 1rem;  
}
```

Categoria Layout

Os estilos de Layout organizam a estrutura principal da página.

```
.l-header {  
    padding: 20px;  
    background-color: #003366;  
    color: white;  
}
```

```
.l-content {  
    max-width: 1200px;  
    margin: 0 auto;  
    padding: 30px;  
}
```

```
.l-footer {  
    padding: 20px;  
    text-align: center;  
    background-color: #f2f2f2;  
}
```

Observe o uso do prefixo `l-`, indicando que essas classes possuem responsabilidade estrutural.

Categoria Module

Os módulos representam componentes reutilizáveis da interface.

```
.news-card {  
    border: 1px solid #cccccc;  
    border-radius: 8px;  
    padding: 20px;  
    margin-bottom: 20px;  
}
```

```
.news-title {  
    margin-top: 15px;
```

```
    margin-bottom: 10px;
}

.news-description {
    margin-bottom: 15px;
}

.button {
    padding: 10px 20px;
    border: none;
    background-color: #0055aa;
    color: white;
    cursor: pointer;
}
```

O componente `news-card` pode ser reutilizado em qualquer área da aplicação sem depender do contexto onde está inserido.

Categoria State

Os estados representam alterações temporárias de comportamento ou aparência.

```
.is-featured {
    border: 3px solid #ff9900;
    background-color: #fff8e6;
}

.is-disabled {
    opacity: 0.5;
    cursor: not-allowed;
}
```

Exemplo de aplicação:

```
<button class="button is-disabled">
  Indisponível
</button>
```

O prefixo `is-` permite identificar rapidamente que a classe representa um estado.

Categoria Theme

Os temas permitem alterar o aspecto visual geral da aplicação.

```
.theme-light {  
    background-color: white;  
    color: #222;  
}
```

```
.theme-dark {  
    background-color: #222;  
    color: white;  
}
```

Aplicação:

```
<body class="theme-dark">
```

A simples troca da classe modifica o tema visual sem necessidade de alterar os módulos ou layouts.

Conclusão

Este exemplo demonstra a principal característica do SMACSS: a separação dos estilos por responsabilidade. Em vez de organizar o CSS apenas em função dos componentes, a metodologia categoriza as regras em Base, Layout, Module, State e Theme, criando uma arquitetura mais organizada e escalável.

Essa abordagem é especialmente útil em projetos de médio e grande porte, onde a manutenção e a evolução das folhas de estilo podem se tornar complexas. Ao adotar o SMACSS, equipes de desenvolvimento conseguem estruturar melhor seus arquivos CSS e reduzir problemas relacionados à duplicação de código e à dificuldade de manutenção.

Referências Bibliográficas

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS Book*. Disponível em: <https://smacss.com/book>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

COYIER, Chris. *Comparing CSS Methodologies*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

VEROU, Lea. *CSS Secrets: Better Solutions to Everyday Web Design Problems*. Sebastopol: O'Reilly Media, 2015.

Exercício Prático – Aplicando a Metodologia SMACSS

Introdução

A metodologia SMACSS propõe a organização do CSS por categorias de responsabilidade, permitindo que projetos web sejam mais escaláveis e fáceis de manter. Para dominar essa abordagem, é fundamental compreender como classificar corretamente os estilos em Base, Layout, Module, State e Theme.

Neste exercício, o estudante deverá analisar um cenário de desenvolvimento real e estruturar os estilos seguindo os princípios da arquitetura SMACSS.

Desenvolvimento

Situação-Problema

Uma instituição de ensino deseja desenvolver um portal acadêmico para seus alunos. A página principal deverá possuir os seguintes elementos:

- Cabeçalho contendo o logotipo da instituição;
- Menu principal de navegação;
- Área de conteúdo com cartões de disciplinas;
- Rodapé institucional.

Cada cartão de disciplina deverá apresentar:

- Nome da disciplina;
- Nome do professor;
- Carga horária;
- Botão para acessar o conteúdo.

Além disso, o sistema deverá suportar:

- Disciplinas em andamento;
- Disciplinas concluídas;
- Disciplinas bloqueadas.

O portal também deverá possuir dois temas visuais:

- Tema claro;
- Tema escuro.

Questão 1

Classifique os seguintes elementos nas categorias do SMACSS:

- body
- Cabeçalho
- Rodapé
- Área principal de conteúdo
- Cartão de disciplina
- Botão de acesso
- Disciplina concluída
- Disciplina bloqueada
- Tema escuro

Organize sua resposta em uma tabela contendo:

Elemento	Categoria SMACSS
----------	------------------

Questão 2

Defina uma nomenclatura adequada para os componentes de Layout.

Crie classes para:

- Cabeçalho
- Conteúdo principal
- Rodapé

Utilize o padrão recomendado pelo SMACSS.

Questão 3

Defina as classes dos módulos necessários para representar um cartão de disciplina.

O componente deve possuir:

- Cartão principal;
 - Nome da disciplina;
 - Nome do professor;
 - Carga horária;
 - Botão de acesso.
-

Questão 4

Crie as classes de estado necessárias para representar:

- Disciplina em andamento;
- Disciplina concluída;
- Disciplina bloqueada.

Utilize os prefixos recomendados pelo SMACSS.

Questão 5

Crie as classes de tema para:

- Tema claro;
- Tema escuro.

Defina propriedades adequadas para alterar:

- Cor de fundo;
 - Cor do texto;
 - Cor dos cartões.
-

Questão 6

Construa a estrutura HTML de uma disciplina concluída utilizando as categorias definidas anteriormente.

A disciplina deve possuir:

- Nome: Desenvolvimento Web Avançado;
 - Professor: Maria Oliveira;
 - Carga horária: 80 horas.
-

Questão 7

Desenvolva o CSS completo para:

- Base;
- Layout;
- Module;
- State;

- Theme.

Utilize boas práticas de organização e indentação.

Questão 8

Explique as vantagens da arquitetura SMACSS em comparação com uma folha de estilos organizada apenas por páginas.

Sua resposta deve abordar:

- Organização;
 - Reutilização;
 - Escalabilidade;
 - Manutenção;
 - Trabalho em equipe.
-

Desafio Proposto

O portal acadêmico será expandido e passará a possuir novos módulos:

- Calendário Acadêmico;
- Avisos;
- Fórum de Discussão;
- Biblioteca Digital.

Para cada módulo:

1. Defina sua classe principal.
2. Crie pelo menos três subcomponentes.
3. Defina dois estados possíveis.
4. Indique quais estilos pertencem às categorias Layout, Module e State.

Conclusão

Este exercício permite praticar todos os conceitos fundamentais do SMACSS em um cenário semelhante aos encontrados em sistemas acadêmicos e aplicações corporativas. Ao classificar corretamente os estilos em categorias de responsabilidade, o estudante desenvolve a capacidade de criar arquiteturas CSS mais organizadas, reutilizáveis e preparadas para crescimento futuro.

A conclusão da atividade deve demonstrar a compreensão dos princípios centrais do SMACSS e sua aplicação prática no desenvolvimento de interfaces modernas.

Referências Bibliográficas

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS Book*. Disponível em: <https://smacss.com/book>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

COYIER, Chris. *Comparing CSS Methodologies*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

KALBACH, James. *Designing Web Navigation*. Sebastopol: O'Reilly Media, 2007.

Resolução do Exercício Prático – Metodologia SMACSS

Introdução

Nesta resolução será aplicada a metodologia SMACSS para estruturar o CSS de um portal acadêmico. O objetivo é demonstrar como classificar estilos em categorias de responsabilidade, promovendo organização, reutilização e escalabilidade do código.

A solução foi elaborada seguindo as recomendações da arquitetura proposta por Jonathan Snook, utilizando as categorias Base, Layout, Module, State e Theme.

Desenvolvimento

Questão 1 – Classificação dos Elementos

Elemento	Categoria SMACSS
body	Base
Cabeçalho	Layout
Rodapé	Layout
Área principal de conteúdo	Layout
Cartão de disciplina	Module
Botão de acesso	Module
Disciplina concluída	State

Elemento	Categoria SMACSS
Disciplina bloqueada	State
Tema escuro	Theme

Questão 2 – Classes de Layout

Seguindo a recomendação do SMACSS, utiliza-se o prefixo `l-` para componentes estruturais.

```
.l-header  
.l-content  
.l-footer
```

Exemplo:

```
.l-header {  
    padding: 20px;  
}  
  
.l-content {  
    max-width: 1200px;  
    margin: 0 auto;  
}  
  
.l-footer {  
    padding: 20px;  
}
```

Questão 3 – Classes dos Módulos

Para representar o cartão de disciplina:

```
.course-card  
.course-title  
.course-professor  
.course-workload  
.course-button
```

Esses componentes podem ser reutilizados em qualquer área do sistema.

Exemplo:

```
.course-card {  
  padding: 20px;  
  border: 1px solid #ccc;  
}
```

```
.course-title {  
  font-size: 1.5rem;  
}
```

```
.course-professor {  
  color: #666;  
}
```

```
.course-workload {  
  font-weight: bold;  
}
```

```
.course-button {  
  padding: 10px 20px;  
}
```

Questão 4 – Classes de Estado

O SMACSS recomenda a utilização do prefixo `is-`.

```
.is-active  
.is-completed  
.is-locked
```

Implementação:

```
.is-active {  
  border-left: 5px solid #0066cc;  
}
```

```
.is-completed {  
  border-left: 5px solid #008000;
```

```
    background-color: #f0fff0;
}

.is-locked {
    opacity: 0.5;
}
```

Questão 5 – Classes de Tema

```
.theme-light
.theme-dark
```

Implementação:

```
.theme-light {
    background-color: #ffffff;
    color: #222222;
}

.theme-light .course-card {
    background-color: #f8f8f8;
}

.theme-dark {
    background-color: #1e1e1e;
    color: #f5f5f5;
}

.theme-dark .course-card {
    background-color: #2d2d2d;
}
```

Questão 6 – Estrutura HTML

```
<body class="theme-light">

    <header class="l-header">
        <h1>Portal Acadêmico</h1>
```

```
</header>
```

```
<main class="l-content">
```

```
  <article class="course-card is-completed">
```

```
    <h2 class="course-title">
```

```
      Desenvolvimento Web Avançado
```

```
    </h2>
```

```
    <p class="course-professor">
```

```
      Professora: Maria Oliveira
```

```
    </p>
```

```
    <p class="course-workload">
```

```
      Carga Horária: 80 horas
```

```
    </p>
```

```
    <button class="course-button">
```

```
      Acessar Conteúdo
```

```
    </button>
```

```
  </article>
```

```
</main>
```

```
<footer class="l-footer">
```

```
  Portal Acadêmico © 2026
```

```
</footer>
```

```
</body>
```

Questão 7 – CSS Completo

Base

```
body {  
  margin: 0;
```

```
    font-family: Arial, sans-serif;
}
```

```
h1,
h2,
p {
    margin: 0;
}
```

Layout

```
.l-header {
    padding: 20px;
    background-color: #003366;
    color: white;
}
```

```
.l-content {
    max-width: 1200px;
    margin: auto;
    padding: 20px;
}
```

```
.l-footer {
    padding: 20px;
    text-align: center;
}
```

Module

```
.course-card {
    border: 1px solid #cccccc;
    border-radius: 8px;
    padding: 20px;
}
```

```
.course-title {
    font-size: 1.5rem;
    margin-bottom: 10px;
}
```

```
.course-professor {  
    margin-bottom: 10px;  
}
```

```
.course-workload {  
    margin-bottom: 15px;  
}
```

```
.course-button {  
    padding: 10px 20px;  
    border: none;  
    background-color: #0055aa;  
    color: white;  
    cursor: pointer;  
}
```

State

```
.is-active {  
    border-left: 5px solid #0055aa;  
}
```

```
.is-completed {  
    border-left: 5px solid green;  
    background-color: #eefdee;  
}
```

```
.is-locked {  
    opacity: 0.5;  
    pointer-events: none;  
}
```

Theme

```
.theme-light {  
    background-color: white;  
    color: #222;  
}
```

```
.theme-dark {  
  background-color: #222;  
  color: white;  
}  
  
.theme-dark .course-card {  
  background-color: #333;  
}
```

Questão 8 – Vantagens do SMACSS

Organização

O SMACSS organiza os estilos por categorias de responsabilidade, facilitando a localização e o entendimento do código.

Reutilização

Módulos podem ser reutilizados em diferentes páginas sem necessidade de duplicar estilos.

Escalabilidade

Novos componentes podem ser adicionados sem impactar a estrutura existente da aplicação.

Manutenção

A separação entre Layout, Module, State e Theme reduz a probabilidade de alterações causarem efeitos indesejados em outras partes do sistema.

Trabalho em Equipe

A categorização padronizada facilita a colaboração entre desenvolvedores, designers e equipes de manutenção.

Desafio Proposto

Calendário Acadêmico

Module

```
.calendar  
.calendar-title  
.calendar-event  
.calendar-date
```

State

.is-current-month
.is-holiday

Avisos

Module

.notice
.notice-title
.notice-content
.notice-date

State

.is-important
.is-expired

Fórum de Discussão

Module

.forum-topic
.forum-author
.forum-message
.forum-reply

State

.is-closed
.is-pinned

Biblioteca Digital

Module

.library-book
.library-title
.library-author
.library-download

State

.is-available

.is-unavailable

Conclusão

A resolução demonstra como a arquitetura SMACSS promove uma separação clara de responsabilidades dentro do CSS. Ao categorizar estilos em Base, Layout, Module, State e Theme, torna-se possível desenvolver aplicações mais organizadas e preparadas para crescimento contínuo.

Essa abordagem é especialmente indicada para projetos de médio e grande porte, nos quais a manutenção e a evolução da interface exigem uma estrutura consistente e facilmente compreensível por toda a equipe de desenvolvimento.

Referências Bibliográficas

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS Book*. Disponível em: <https://smacss.com/book>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

COYIER, Chris. *Comparing CSS Methodologies*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

VEROU, Lea. *CSS Secrets: Better Solutions to Everyday Web Design Problems*. Sebastopol: O'Reilly Media, 2015.

OOCSS

Metodologia OOCSS

Introdução

OOCSS (*Object-Oriented CSS*) é uma metodologia de organização de código CSS criada por Nicole Sullivan com o objetivo de promover reutilização, modularidade e manutenção eficiente das folhas de estilo. Inspirada nos princípios da orientação a objetos da engenharia de software, essa abordagem busca transformar elementos visuais da interface em componentes independentes e reutilizáveis.

A principal proposta do OOCSS é evitar a repetição de código e reduzir o acoplamento entre estrutura e aparência. Em vez de criar estilos específicos para cada elemento da interface, a metodologia incentiva a construção de objetos genéricos que podem ser combinados para formar componentes mais complexos.

Desenvolvimento

O OOCSS é fundamentado em dois princípios centrais: **separação entre estrutura e aparência** e **separação entre contêiner e conteúdo**. Esses conceitos orientam a criação de componentes flexíveis e reutilizáveis.

Separação entre Estrutura e Aparência

O primeiro princípio recomenda que características estruturais sejam separadas das características visuais.

Por exemplo, em vez de criar uma única classe contendo dimensões, posicionamento e cores:

```
.card-azul {  
    width: 400px;  
    padding: 20px;  
    background-color: blue;  
    color: white;  
}
```

o OOCSS sugere dividir essas responsabilidades:

```
.box {  
    width: 400px;  
    padding: 20px;  
}
```

```
.bg-blue {  
  background-color: blue;  
  color: white;  
}
```

Aplicação:

```
<div class="box bg-blue">  
  Conteúdo  
</div>
```

Dessa forma, a mesma estrutura pode ser reutilizada com diferentes estilos visuais.

Separação entre Contêiner e Conteúdo

O segundo princípio recomenda que os estilos de um componente não dependam do local onde ele está inserido.

Uma abordagem não recomendada seria:

```
.sidebar .titulo {  
  font-size: 1.5rem;  
}
```

Nesse caso, o comportamento do título depende diretamente do contêiner.

No OOCSS, a preferência é por classes independentes:

```
.titulo {  
  font-size: 1.5rem;  
}
```

Assim, o mesmo componente pode ser reutilizado em qualquer contexto da aplicação.

Objetos Reutilizáveis

Os componentes são construídos como objetos genéricos que podem ser combinados.

Exemplo:

```
.media {  
  display: flex;  
  gap: 20px;  
}
```

```
.media-image {  
  width: 150px;  
}
```

```
.media-content {  
  flex: 1;  
}
```

HTML:

```
<div class="media">  
  
    
  
  <div class="media-content">  
    <h2>Título</h2>  
    <p>Descrição</p>  
  </div>  
  
</div>
```

O objeto `media` pode ser utilizado para produtos, notícias, usuários, artigos e diversos outros componentes.

Composição de Classes

Uma característica marcante do OOCSS é a composição de múltiplas classes para formar um componente.

Exemplo:

```
.box {  
  padding: 20px;  
}  
  
.rounded {  
  border-radius: 8px;  
}  
  
.shadow {  
  box-shadow: 0 2px 8px rgba(0,0,0,.2);  
}
```

```
.bg-light {  
    background-color: #f5f5f5;  
}
```

HTML:

```
<div class="box rounded shadow bg-light">  
    Conteúdo do componente  
</div>
```

Cada classe possui uma responsabilidade específica e pode ser reutilizada em diversos contextos.

Benefícios da Abordagem

A adoção do OOCSS oferece diversas vantagens:

- Redução da duplicação de código;
- Maior reutilização de componentes;
- Menor quantidade de CSS necessário;
- Facilidade de manutenção;
- Melhor escalabilidade em projetos grandes;
- Maior consistência visual da interface.

Esses benefícios tornaram o OOCSS uma das metodologias que influenciaram diversas arquiteturas modernas de CSS, incluindo abordagens baseadas em componentes utilizadas atualmente.

Conclusão

O OOCSS introduziu uma mudança importante na forma de organizar folhas de estilo ao aplicar conceitos da orientação a objetos ao desenvolvimento front-end. Sua proposta de separar estrutura e aparência, bem como tornar componentes independentes de seu contexto, contribui para a criação de sistemas CSS mais modulares e reutilizáveis.

Embora metodologias mais recentes tenham surgido posteriormente, muitos dos princípios do OOCSS continuam presentes em frameworks modernos e em arquiteturas CSS contemporâneas. Por esse motivo, seu estudo permanece relevante para a compreensão da evolução das boas práticas no desenvolvimento de interfaces web.

Referências Bibliográficas

SULLIVAN, Nicole. *Object Oriented CSS*. Disponível em: <http://oocss.org>. Acesso em: 25 maio 2026.

SULLIVAN, Nicole; APELLO, Jonathan. *The Object-Oriented CSS Approach*. Disponível em: <http://oocss.org>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

COYIER, Chris. *A Look at OOCSS*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

Sintaxe da Metodologia OOCSS

Introdução

Diferentemente de metodologias como BEM e SMACSS, o OOCSS não define uma convenção rígida de nomenclatura para classes CSS. Sua principal preocupação não está na forma como as classes são nomeadas, mas sim na maneira como os estilos são estruturados e reutilizados.

A sintaxe do OOCSS é baseada na criação de pequenos objetos independentes, cada um responsável por uma característica específica da interface. Esses objetos podem ser combinados para formar componentes mais complexos, reduzindo a duplicação de código e aumentando a reutilização dos estilos.

Desenvolvimento

Classes Estruturais

As classes estruturais definem características relacionadas à organização física do componente, como largura, espaçamento, alinhamento e posicionamento.

Sintaxe:

```
.nome-estrutura {  
    propriedade: valor;  
}
```

Exemplo:

```
.box {  
    width: 400px;  
    padding: 20px;  
}
```

```
.container {  
    max-width: 1200px;
```

```
    margin: 0 auto;
}
```

```
.flex {
    display: flex;
}
```

Essas classes podem ser utilizadas em diferentes componentes da aplicação.

Classes de Aparência

As classes de aparência definem características visuais como cores, bordas, sombras e tipografia.

Sintaxe:

```
.nome-aparencia {
    propriedade: valor;
}
```

Exemplo:

```
.bg-primary {
    background-color: #0055aa;
}
```

```
.bg-success {
    background-color: #28a745;
}
```

```
.text-white {
    color: white;
}
```

```
.shadow {
    box-shadow: 0 2px 8px rgba(0,0,0,.2);
}
```

A aparência é separada da estrutura para aumentar a reutilização dos estilos.

Composição de Classes

Uma das características mais importantes do OOCSS é a composição de múltiplas classes em um único elemento.

HTML:

```
<div class="box bg-primary text-white shadow">  
  Conteúdo  
</div>
```

Nesse exemplo:

- `box` define a estrutura;
- `bg-primary` define a cor de fundo;
- `text-white` define a cor do texto;
- `shadow` adiciona sombra.

Cada classe possui responsabilidade única e pode ser reutilizada em diversos componentes.

Objetos de Interface

O OOCSS incentiva a criação de objetos reutilizáveis para componentes recorrentes.

Exemplo: Objeto de mídia

```
.media {  
  display: flex;  
  gap: 20px;  
}
```

```
.media-image {  
  width: 200px;  
}
```

```
.media-body {  
  flex: 1;  
}
```

HTML:

```
<div class="media">  
  
  
```

```
<div class="media-body">
```

```
<h2>Notebook Gamer</h2>
```

```
<p>
    Equipamento de alto desempenho para
    desenvolvimento e modelagem 3D.
</p>
```

```
</div>
```

```
</div>
```

O mesmo objeto pode ser reutilizado em notícias, produtos, perfis de usuários ou artigos.

Classes Utilitárias

O OOCSS frequentemente utiliza classes utilitárias para representar pequenas funcionalidades reutilizáveis.

Exemplo:

```
.mt-10 {
    margin-top: 10px;
}
```

```
.mb-20 {
    margin-bottom: 20px;
}
```

```
.text-center {
    text-align: center;
}
```

```
.hidden {
    display: none;
}
```

```
}
```

Aplicação:

```
<h2 class="text-center mb-20">  
  Relatório de Vendas  
</h2>
```

Essas classes reduzem a necessidade de criar estilos específicos para cada componente.

Exemplo Completo

CSS

```
.box {  
  padding: 20px;  
}  
  
.rounded {  
  border-radius: 8px;  
}  
  
.shadow {  
  box-shadow: 0 2px 8px rgba(0,0,0,.15);  
}  
  
.bg-light {  
  background-color: #f5f5f5;  
}  
  
.text-dark {  
  color: #222;  
}  
  
.flex {  
  display: flex;  
}  
  
.flex-column {  
  flex-direction: column;
```

```
}
```

HTML

```
<div class="box rounded shadow bg-light">

    <h2 class="text-dark">
        Desenvolvimento Web
    </h2>

    <p class="text-dark">
        Disciplina voltada para o desenvolvimento
        de aplicações modernas para a Web.
    </p>

</div>
```

Observe que nenhuma classe foi criada especificamente para esse componente. O elemento foi construído a partir da combinação de objetos reutilizáveis.

Conclusão

A sintaxe do OOCSS é baseada na composição de classes independentes e reutilizáveis. Em vez de criar componentes altamente específicos, a metodologia incentiva a separação entre estrutura e aparência, permitindo que diferentes objetos sejam combinados para produzir interfaces variadas.

Essa abordagem reduz a duplicação de código, facilita a manutenção e promove a reutilização de estilos em projetos de médio e grande porte. Embora não possua uma convenção formal de nomenclatura, sua filosofia influenciou diversas metodologias e frameworks modernos de desenvolvimento front-end.

Referências Bibliográficas

SULLIVAN, Nicole. *Object Oriented CSS*. Disponível em: <http://oocss.org>. Acesso em: 25 maio 2026.

SULLIVAN, Nicole. *OOCSS Wiki*. Disponível em: <https://github.com/stubbornella/oocss/wiki>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

COYIER, Chris. *A Look at OOCSS*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

Exemplo Prático da Metodologia OOCSS

Introdução

A metodologia OOCSS (Object-Oriented CSS) busca construir interfaces por meio da combinação de objetos reutilizáveis. Em vez de criar estilos específicos para cada componente, o desenvolvedor cria classes independentes para estrutura, aparência e comportamento visual, que podem ser reutilizadas em diferentes partes da aplicação.

Neste exemplo será desenvolvido um componente de cartão de curso para um portal acadêmico. O objetivo é demonstrar como os princípios do OOCSS podem ser aplicados para reduzir a duplicação de código e aumentar a reutilização dos estilos.

Desenvolvimento

Cenário

Um portal acadêmico exibe diversos cursos disponíveis para matrícula. Cada cartão deve apresentar:

- Nome do curso;
- Descrição;
- Carga horária;
- Botão de inscrição.

Além disso, alguns cartões podem possuir diferentes aparências, como destaque, bordas especiais ou cores alternativas.

Estrutura dos Objetos CSS

Primeiramente, serão criados objetos reutilizáveis para estrutura.

```
.box {  
    padding: 20px;  
}  
  
.flex-column {  
    display: flex;  
    flex-direction: column;  
}  
  
.full-width {  
    width: 100%;  
}
```

```
.rounded {  
  border-radius: 8px;  
}
```

Agora serão definidos objetos de aparência.

```
.bg-white {  
  background-color: white;  
}
```

```
.bg-highlight {  
  background-color: #eef7ff;  
}
```

```
.text-dark {  
  color: #222;  
}
```

```
.border {  
  border: 1px solid #cccccc;  
}
```

```
.border-primary {  
  border: 2px solid #0055aa;  
}
```

```
.shadow {  
  box-shadow: 0 2px 8px rgba(0,0,0,.15);  
}
```

Também serão criados objetos para tipografia.

```
.title {  
  font-size: 1.5rem;  
  font-weight: bold;  
}
```

```
.subtitle {  
  font-size: 1rem;  
  color: #666;  
}
```

```
.text {  
  line-height: 1.6;  
}
```

Por fim, serão criados objetos para botões.

```
.button {  
  padding: 12px 20px;  
  border: none;  
  cursor: pointer;  
}
```

```
.button-primary {  
  background-color: #0055aa;  
  color: white;  
}
```

```
.button-success {  
  background-color: #198754;  
  color: white;  
}
```

Estrutura HTML

Com os objetos definidos, o componente pode ser construído pela composição de classes.

```
<article  
  class="box rounded border shadow bg-white flex-column">  
  
  <h2 class="title text-dark">  
    Desenvolvimento Web Avançado  
  </h2>  
  
  <p class="subtitle">  
    Carga Horária: 80 horas  
  </p>  
  
  <p class="text text-dark">  
    Estudo de arquiteturas modernas para  
    aplicações web, integração com APIs
```

e otimização de interfaces.

```
<button class="button button-primary">  
  Inscrever-se  
</button>
```

```
</article>
```

Observe que nenhuma classe específica chamada `curso-card` foi criada. O componente é formado pela combinação de objetos reutilizáveis.

Criando uma Variação do Componente

Suponha que alguns cursos precisem ser destacados.

Nesse caso, basta reutilizar os objetos existentes.

```
<article  
  class="box rounded border-primary shadow bg-highlight flex-column">  
  
  <h2 class="title text-dark">  
    Inteligência Artificial Aplicada  
  </h2>  
  
  <p class="subtitle">  
    Carga Horária: 120 horas  
  </p>  
  
  <p class="text text-dark">  
    Técnicas de aprendizado de máquina,  
    redes neurais e análise de dados.  
  </p>  
  
  <button class="button button-success">  
    Inscrever-se  
  </button>  
  
</article>
```

Nenhum CSS adicional precisou ser criado. Apenas diferentes objetos foram combinados.

Comparação com uma Abordagem Tradicional

Sem OOCSS, seria comum criar componentes específicos:

```
.curso-card {  
    padding: 20px;  
    border-radius: 8px;  
    border: 1px solid #ccc;  
    background-color: white;  
}
```

```
.curso-card-destaque {  
    padding: 20px;  
    border-radius: 8px;  
    border: 2px solid #0055aa;  
    background-color: #eef7ff;  
}
```

Nesse cenário, várias propriedades são repetidas.

Com OOCSS:

```
.box {}  
.rounded {}  
.border {}  
.border-primary {}  
.bg-white {}  
.bg-highlight {}
```

Os estilos são reutilizados e a duplicação é reduzida significativamente.

Conclusão

Este exemplo demonstra a essência do OOCSS: construir interfaces por meio da composição de objetos independentes. A separação entre estrutura e aparência permite que componentes sejam montados utilizando classes reutilizáveis, reduzindo a quantidade de CSS necessária e facilitando a manutenção do projeto.

Em aplicações de médio e grande porte, essa abordagem contribui para a padronização visual, melhora a reutilização de código e reduz o esforço necessário para criar novas variações de componentes.

Referências Bibliográficas

SULLIVAN, Nicole. *Object Oriented CSS*. Disponível em: <http://oocss.org>. Acesso em: 25 maio 2026.

SULLIVAN, Nicole. *OOCSS Wiki*. Disponível em: <https://github.com/stubbornella/oocss/wiki>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

COYIER, Chris. *A Look at OOCSS*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

Exercício Prático – Aplicando a Metodologia OOCSS

Introdução

A metodologia OOCSS (Object-Oriented CSS) propõe a construção de interfaces por meio da reutilização de objetos independentes. Em vez de criar estilos específicos para cada componente, o desenvolvedor cria classes genéricas responsáveis por características estruturais e visuais que podem ser combinadas em diferentes contextos.

Neste exercício, o estudante deverá aplicar os princípios do OOCSS para desenvolver componentes de um sistema de gerenciamento de eventos acadêmicos, praticando a separação entre estrutura e aparência e a composição de objetos reutilizáveis.

Desenvolvimento

Situação-Problema

Uma universidade está desenvolvendo um portal para divulgação de eventos acadêmicos. O sistema deve exibir cartões contendo informações sobre palestras, workshops e minicursos.

Cada cartão deve apresentar:

- Título do evento;
- Nome do palestrante;
- Data do evento;
- Descrição resumida;
- Botão de inscrição.

O sistema deverá permitir diferentes variações visuais:

- Evento padrão;
- Evento em destaque;

- Evento encerrado.

A equipe de desenvolvimento decidiu utilizar OOCSS para maximizar a reutilização dos estilos.

Questão 1

Identifique quais características pertencem à estrutura e quais pertencem à aparência.

Classifique os seguintes itens:

- Espaçamento interno;
- Cor de fundo;
- Cor do texto;
- Largura do cartão;
- Sombra;
- Borda;
- Alinhamento dos elementos;
- Cor do botão;
- Arredondamento dos cantos.

Organize a resposta em uma tabela.

Característica	Estrutura ou Aparência
----------------	------------------------

Questão 2

Crie classes estruturais para representar:

- Espaçamento interno;
- Largura do cartão;
- Organização vertical dos elementos;
- Alinhamento centralizado.

Utilize nomes coerentes com a filosofia OOCSS.

Questão 3

Crie classes de aparência para representar:

- Fundo branco;
- Fundo destacado;

- Texto escuro;
 - Texto claro;
 - Borda padrão;
 - Borda de destaque;
 - Sombra.
-

Questão 4

Crie classes para os botões de inscrição.

O sistema deve possuir:

- Botão padrão;
- Botão de destaque;
- Botão desabilitado.

Separe corretamente estrutura e aparência.

Questão 5

Monte o HTML de um cartão de evento utilizando exclusivamente a composição de objetos OOCSS.

Dados:

- Evento: Congresso de Engenharia de Software;
 - Palestrante: Carlos Almeida;
 - Data: 15/08/2026;
 - Tipo: Evento em destaque.
-

Questão 6

Explique por que a seguinte implementação não segue os princípios do OOCSS:

```
.evento-destaque {  
  width: 400px;  
  padding: 20px;  
  background-color: #eef7ff;  
  border: 2px solid #0055aa;  
  border-radius: 8px;
```

}

Sua resposta deve abordar:

- Reutilização;
 - Acoplamento;
 - Separação de responsabilidades.
-

Questão 7

Reescreva o exemplo da questão anterior utilizando corretamente a metodologia OOCSS.

Questão 8

Considere que o sistema agora precisa exibir também:

- Notícias;
- Cursos;
- Projetos de Pesquisa.

Explique como os objetos já criados para os eventos poderiam ser reutilizados nesses novos componentes.

Apresente exemplos de composição de classes.

Desafio Proposto

Desenvolva uma biblioteca de objetos reutilizáveis para um portal universitário contendo:

Objetos Estruturais

- Container;
- Grid;
- Flexbox;
- Espaçamento;
- Alinhamento.

Objetos Visuais

- Fundos;
- Bordas;
- Sombras;

- Tipografia;
- Botões.

Para cada objeto:

1. Defina o nome da classe.
2. Explique sua responsabilidade.
3. Apresente um exemplo de uso.
4. Indique quais componentes poderiam reutilizá-lo.

Conclusão

Este exercício permite praticar os dois princípios fundamentais do OOCSS: a separação entre estrutura e aparência e a criação de objetos reutilizáveis. A correta aplicação desses conceitos contribui para a construção de sistemas CSS mais flexíveis, reduzindo a duplicação de código e facilitando a manutenção da aplicação.

Ao concluir a atividade, o estudante deverá ser capaz de identificar oportunidades de reutilização, projetar objetos independentes e construir componentes complexos por meio da composição de classes.

Referências Bibliográficas

SULLIVAN, Nicole. *Object Oriented CSS*. Disponível em: <http://oocss.org>. Acesso em: 25 maio 2026.

SULLIVAN, Nicole. *OOCSS Wiki*. Disponível em: <https://github.com/stubbornella/oocss/wiki>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

COYIER, Chris. *A Look at OOCSS*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

Resolução do Exercício Prático – Metodologia OOCSS

Introdução

Nesta resolução será aplicada a metodologia OOCSS para modelar componentes de um portal de eventos acadêmicos. O foco será demonstrar a separação entre estrutura e aparência, bem como a reutilização de objetos independentes para compor diferentes componentes da interface.

A solução busca evidenciar como o OOCSS reduz a duplicação de código e facilita a manutenção de projetos de médio e grande porte.

Desenvolvimento

Questão 1 – Classificação das Características

Característica	Estrutura ou Aparência
Espaçamento interno	Estrutura
Cor de fundo	Aparência
Cor do texto	Aparência
Largura do cartão	Estrutura
Sombra	Aparência
Borda	Aparência
Alinhamento dos elementos	Estrutura
Cor do botão	Aparência
Arredondamento dos cantos	Aparência

Justificativa:

Características estruturais definem organização, posicionamento e dimensões. Características de aparência definem a identidade visual do componente.

Questão 2 – Classes Estruturais

```
.container {  
  width: 100%;  
}  
  
.card-width {  
  max-width: 500px;  
}  
  
.padding-lg {  
  padding: 24px;  
}
```

```
.flex-column {  
    display: flex;  
    flex-direction: column;  
}
```

```
.center-content {  
    align-items: center;  
}
```

Essas classes podem ser reutilizadas em eventos, notícias, cursos ou qualquer outro componente do sistema.

Questão 3 – Classes de Aparência

```
.bg-white {  
    background-color: #ffffff;  
}
```

```
.bg-highlight {  
    background-color: #eef7ff;  
}
```

```
.text-dark {  
    color: #222222;  
}
```

```
.text-light {  
    color: #ffffff;  
}
```

```
.border {  
    border: 1px solid #cccccc;  
}
```

```
.border-highlight {  
    border: 2px solid #0055aa;  
}
```

```
.shadow {  
  box-shadow: 0 2px 8px rgba(0,0,0,.15);  
}
```

```
.rounded {  
  border-radius: 8px;  
}
```

Observe que nenhuma dessas classes depende de um componente específico.

Questão 4 – Classes dos Botões

Estrutura

```
.button {  
  padding: 12px 20px;  
  border: none;  
  cursor: pointer;  
}
```

Aparência

```
.button-primary {  
  background-color: #0055aa;  
  color: white;  
}
```

```
.button-highlight {  
  background-color: #ff9900;  
  color: white;  
}
```

```
.button-disabled {  
  background-color: #999999;  
  color: white;  
  cursor: not-allowed;  
}
```

Utilização:

```
<button class="button button-primary">
```

Inscrever-se
</button>

Questão 5 – HTML do Evento

```
<article
  class="
    card-width
    padding-lg
    flex-column
    bg-highlight
    border-highlight
    rounded
    shadow">

  <h2 class="text-dark">
    Congresso de Engenharia de Software
  </h2>

  <p class="text-dark">
    Palestrante: Carlos Almeida
  </p>

  <p class="text-dark">
    Data: 15/08/2026
  </p>

  <p class="text-dark">
    Evento voltado à discussão de práticas
    modernas de desenvolvimento de software.
  </p>

  <button class="button button-highlight">
    Inscrever-se
  </button>

</article>
```

O componente é construído exclusivamente pela combinação de objetos reutilizáveis.

Questão 6 – Análise do Código Incorreto

Código apresentado:

```
.evento-destaque {  
    width: 400px;  
    padding: 20px;  
    background-color: #eef7ff;  
    border: 2px solid #0055aa;  
    border-radius: 8px;  
}
```

Reutilização

O código concentra diversas responsabilidades em uma única classe. Caso outro componente necessite apenas da borda ou do espaçamento, será necessário duplicar estilos.

Acoplamento

A classe está fortemente vinculada ao componente "evento", dificultando sua reutilização em notícias, cursos ou outros elementos.

Separação de Responsabilidades

A classe mistura características estruturais (largura e espaçamento) com características visuais (cor de fundo e borda), contrariando um dos princípios fundamentais do OOCSS.

Questão 7 – Reescrita Utilizando OOCSS

CSS

```
.card-width {  
    width: 400px;  
}  
  
.padding-lg {  
    padding: 20px;  
}  
  
.bg-highlight {  
    background-color: #eef7ff;
```

```
}
```

```
.border-highlight {  
  border: 2px solid #0055aa;  
}
```

```
.rounded {  
  border-radius: 8px;  
}
```

HTML

```
<div  
  class="  
    card-width  
    padding-lg  
    bg-highlight  
    border-highlight  
    rounded">  
</div>
```

Agora cada objeto possui uma única responsabilidade e pode ser reutilizado em diferentes componentes.

Questão 8 – Reutilização em Outros Componentes

Notícia

```
<article  
  class="  
    card-width  
    padding-lg  
    bg-white  
    border  
    rounded  
    shadow">  
  
  <h2 class="text-dark">  
    Nova Biblioteca Digital  
  </h2>
```

</article>

Curso

```
<article
  class="
    card-width
    padding- lg
    bg-highlight
    rounded">

  <h2 class="text-dark">
    Desenvolvimento Web Avançado
  </h2>
```

</article>

Projeto de Pesquisa

```
<article
  class="
    card-width
    padding- lg
    border
    shadow
    rounded">

  <h2 class="text-dark">
    Inteligência Artificial Aplicada
  </h2>
```

</article>

Os mesmos objetos são reutilizados sem necessidade de criar novas regras CSS.

Desafio Proposto

Objetos Estruturais

Container

```
.container {  
    max-width: 1200px;  
    margin: 0 auto;  
}
```

Responsabilidade: limitar a largura da página.

Grid

```
.grid {  
    display: grid;  
    gap: 20px;  
}
```

Responsabilidade: organização em grade.

Flexbox

```
.flex {  
    display: flex;  
}
```

Responsabilidade: alinhamento flexível.

Espaçamento

```
.padding-lg {  
    padding: 24px;  
}  
  
.margin-lg {  
    margin: 24px;  
}
```

Responsabilidade: espaçamento interno e externo.

Alinhamento

```
.center-content {  
  align-items: center;  
  justify-content: center;  
}
```

Responsabilidade: centralização.

Objetos Visuais

Fundos

```
.bg-white {}  
.bg-light {}  
.bg-primary {}
```

Bordas

```
.border {}  
.border-highlight {}
```

Sombras

```
.shadow {}  
.shadow-lg {}
```

Tipografia

```
.text-dark {}  
.text-light {}  
.title {}  
.subtitle {}
```

Botões

```
.button {}  
.button-primary {}  
.button-success {}
```

```
.button-disabled {}
```

Exemplos de Reutilização

Objeto	Componentes que podem reutilizá-lo
container	Página inicial, relatórios, portal acadêmico
grid	Galeria, catálogo de cursos, lista de eventos
flex	Menus, cabeçalhos, cartões
shadow	Notícias, cursos, eventos
button-primary	Formulários, inscrições, login
bg-light	Cards, painéis, dashboards
title	Notícias, eventos, disciplinas

Conclusão

A resolução evidencia que o OOCSS não se concentra na criação de componentes altamente específicos, mas sim na construção de objetos reutilizáveis que podem ser combinados para formar diferentes interfaces. A separação entre estrutura e aparência reduz o acoplamento entre estilos e componentes, favorecendo a reutilização e a manutenção do código.

Em projetos maiores, essa abordagem contribui para a redução da duplicação de regras CSS, melhora a consistência visual da aplicação e facilita a evolução do sistema ao longo do tempo.

Referências Bibliográficas

SULLIVAN, Nicole. *Object Oriented CSS*. Disponível em: <http://oocss.org>. Acesso em: 25 maio 2026.

SULLIVAN, Nicole. *OOCSS Wiki*. Disponível em: <https://github.com/stubbornella/oocss/wiki>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

COYIER, Chris. *A Look at OOCSS*. Disponível em: <https://css-tricks.com>. Acesso em: 25 maio 2026.

SUITCSS

Metodologia SUIT CSS

Introdução

SUIT CSS (*Systematic User Interface Toolkit CSS*) é uma metodologia de arquitetura CSS criada para promover a construção de interfaces modulares, previsíveis e escaláveis. Seu principal objetivo é estabelecer um padrão consistente de nomenclatura e organização de componentes, facilitando a manutenção e a colaboração em projetos de médio e grande porte.

A metodologia foi desenvolvida com forte influência dos conceitos de CSS modular e arquitetura baseada em componentes. Diferentemente de abordagens mais flexíveis, como OOCSS e SMACSS, o SUIT CSS define convenções rígidas para a nomenclatura das classes, permitindo identificar claramente componentes, elementos internos, estados e modificadores.

Desenvolvimento

A arquitetura SUIT CSS é baseada na ideia de que cada componente da interface deve ser tratado como uma unidade independente. Para isso, estabelece padrões específicos para nomeação de componentes, elementos, modificadores e estados.

Componentes

Os componentes representam blocos independentes da interface. A nomenclatura utiliza o padrão *PascalCase* (também chamado de Upper Camel Case), em que cada palavra inicia com letra maiúscula.

Exemplos:

```
.Button {}  
.Card {}  
.Navigation {}  
.SearchForm {}  
.ProductList {}
```

Essa convenção facilita a distinção entre componentes e outras classes utilitárias.

Descendentes

Elementos internos de um componente são chamados de descendentes. A relação entre componente e descendente é representada por um hífen simples (-).

Exemplos:

```
.Card-title {}
```

```
.Card-description {}  
.Card-image {}
```

```
.Navigation-item {}  
.Navigation-link {}
```

Nesse modelo, os elementos permanecem claramente associados ao componente ao qual pertencem.

Modificadores

Os modificadores representam variações visuais ou funcionais de um componente. O SUIT CSS utiliza dois hífen (- -) para indicar modificações.

Exemplos:

```
.Button--primary {}  
.Button--secondary {}  
  
.Card--featured {}  
.Card--compact {}
```

Os modificadores permitem criar diferentes versões de um mesmo componente sem alterar sua estrutura principal.

Estados

Os estados representam condições temporárias de um componente. A metodologia utiliza o prefixo `is-`.

Exemplos:

```
.is-active {}  
.is-disabled {}  
.is-hidden {}  
.is-expanded {}
```

Aplicação:

```
<button class="Button is-disabled">  
  Salvar  
</button>
```

Essa convenção facilita a identificação de estilos relacionados ao comportamento dinâmico da interface.

Utilitários

O SUIT CSS também prevê o uso de classes utilitárias para tarefas específicas e reutilizáveis. Essas classes recebem o prefixo `u-`.

Exemplos:

```
.u-textCenter {}  
.u-hidden {}  
.u-marginTopLarge {}  
.u-flex {}
```

Aplicação:

```
<div class="u-textCenter">  
    Conteúdo Centralizado  
</div>
```

As classes utilitárias possuem responsabilidade única e podem ser utilizadas em diversos componentes.

Vantagens da Metodologia

A adoção do SUIT CSS oferece diversos benefícios:

- Padronização rigorosa da nomenclatura;
- Facilidade de leitura do código;
- Modularização dos componentes;
- Redução de conflitos entre estilos;
- Melhor integração com equipes de desenvolvimento;
- Facilidade de escalabilidade em aplicações grandes.

Além disso, sua estrutura favorece o desenvolvimento orientado a componentes, abordagem amplamente utilizada em frameworks modernos.

Conclusão

O SUIT CSS é uma metodologia que combina os princípios da modularização com uma convenção rigorosa de nomenclatura. Sua proposta é criar componentes independentes e reutilizáveis, tornando o CSS mais previsível e fácil de manter.

Ao utilizar padrões específicos para componentes, descendentes, modificadores, estados e utilitários, a metodologia promove uma organização consistente das folhas de estilo. Por esse motivo, o SUIT CSS é frequentemente adotado em projetos que exigem alto grau de padronização e colaboração entre equipes de desenvolvimento.

Referências Bibliográficas

NICOLE SULLIVAN et al. *SUIT CSS Documentation*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SUIT CSS. *Naming Conventions*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

WILLIAMS, Alex. *CSS Architecture and Design Systems*. New York: Apress, 2021.

Sintaxe da Metodologia SUIT CSS

Introdução

A metodologia SUIT CSS define uma convenção de nomenclatura rígida para componentes de interface. Seu objetivo é criar uma arquitetura CSS consistente, previsível e escalável, permitindo que os desenvolvedores identifiquem facilmente a função de cada classe apenas observando seu nome.

A sintaxe do SUIT CSS é baseada em cinco elementos principais: **Componentes**, **Descendentes**, **Modificadores**, **Estados** e **Utilitários**. Cada um possui uma convenção específica que deve ser seguida para garantir a padronização do código.

Desenvolvimento

Componentes

Os componentes representam blocos independentes da interface. A nomenclatura utiliza o padrão **PascalCase**, no qual cada palavra inicia com letra maiúscula.

Sintaxe:

```
.ComponentName {}
```

Exemplos:

```
.Button {}  
.Card {}  
.Navigation {}  
.SearchForm {}  
.ProductList {}
```

Classes de componentes devem possuir nomes descritivos que representem claramente sua responsabilidade dentro da interface.

Descendentes

Os descendentes representam elementos internos de um componente.

A associação entre componente e descendente é realizada por meio de um hífen simples (-).

Sintaxe:

```
.ComponentName-descendantName {}
```

Exemplos:

```
.Card-title {}
```

```
.Card-description {}
```

```
.Card-image {}
```

```
.Navigation-item {}
```

```
.Navigation-link {}
```

```
.SearchForm-input {}
```

```
.SearchForm-button {}
```

HTML:

```
<article class="Card">
```

```
  
```

```
  <h2 class="Card-title">
    Notebook Gamer
  </h2>
```

```
  <p class="Card-description">
    Equipamento para aplicações de alto desempenho.
  </p>
```

```
</article>
```

Os descendentes devem ser utilizados apenas dentro do componente ao qual pertencem.

Modificadores

Os modificadores representam variações de aparência ou comportamento de um componente.

A nomenclatura utiliza dois hífen (- -).

Sintaxe:

```
.ComponentName--modifierName {}
```

Exemplos:

```
.Button--primary {}
```

```
.Button--secondary {}
```

```
.Card--featured {}
```

```
.Card--compact {}
```

```
.Navigation--horizontal {}
```

```
.Navigation--vertical {}
```

HTML:

```
<button class="Button Button--primary">
  Salvar
</button>
```

Outro exemplo:

```
<article class="Card Card--featured">
  ...
</article>
```

O componente mantém sua estrutura original e recebe características adicionais por meio do modificador.

Estados

Os estados representam condições temporárias do componente.

A nomenclatura utiliza o prefixo `is-`.

Sintaxe:

```
.is-stateName {}
```

Exemplos:

```
.is-active {}  
.is-disabled {}  
.is-hidden {}  
.is-expanded {}  
.is-loading {}
```

CSS:

```
.is-disabled {  
    opacity: 0.5;  
    pointer-events: none;  
}
```

```
.is-hidden {  
    display: none;  
}
```

HTML:

```
<button  
    class="Button Button--primary is-disabled">  
    Salvar  
</button>
```

Os estados normalmente são manipulados por JavaScript durante a execução da aplicação.

Utilitários

Os utilitários são classes responsáveis por tarefas específicas e reutilizáveis.

A nomenclatura utiliza o prefixo `u-`.

Sintaxe:

```
.u-utilityName {}
```

Exemplos:

```
.u-textCenter {}  
.u-hidden {}  
.u-flex {}
```

```
.u-marginTopLarge {}  
.u-paddingMedium {}
```

CSS:

```
.u-textCenter {  
    text-align: center;  
}
```

```
.u-hidden {  
    display: none;  
}
```

```
.u-flex {  
    display: flex;  
}
```

HTML:

```
<div class="u-textCenter">  
    Texto Centralizado  
</div>
```

Os utilitários possuem responsabilidade única e podem ser utilizados em qualquer componente.

Variáveis CSS (Opcional)

Embora não faça parte da especificação original do SUIT CSS, é comum utilizar variáveis CSS para aumentar a consistência visual.

```
:root {  
    --primary-color: #0055aa;  
    --secondary-color: #28a745;  
}
```

Aplicação:

```
.Button--primary {  
    background-color: var(--primary-color);  
}
```

Exemplo Completo

HTML

```
<header class="Navigation Navigation--horizontal">

  <ul>

    <li class="Navigation-item">
      <a
        class="Navigation-link is-active"
        href="#">
        Início
      </a>
    </li>

    <li class="Navigation-item">
      <a
        class="Navigation-link"
        href="#">
        Cursos
      </a>
    </li>

  </ul>

</header>

<article class="Card Card--featured">

  

  <h2 class="Card-title">
    Desenvolvimento Web
  </h2>

  <p class="Card-description">
```

Curso voltado ao desenvolvimento
de aplicações modernas.

</p>

<button
 class="Button Button--primary">
 Inscrever-se
</button>

</article>

CSS

.Navigation {}

.Navigation-item {}

.Navigation-link {}

.Navigation--horizontal {}

.Card {}

.Card-image {}

.Card-title {}

.Card-description {}

.Card--featured {}

.Button {}

.Button--primary {}

.is-active {}

.u-textCenter {}

Nesse exemplo é possível identificar claramente cada categoria da metodologia apenas observando os nomes das classes.

Conclusão

A sintaxe do SUIT CSS foi projetada para criar uma arquitetura altamente previsível e padronizada. A utilização de PascalCase para componentes, hífen para descendentes, modificadores com `-`, estados com `is-` e utilitários com `u-` facilita a leitura e a manutenção do código.

Essa abordagem é especialmente útil em aplicações de médio e grande porte, onde a consistência da nomenclatura desempenha papel fundamental na colaboração entre equipes e na evolução contínua da interface.

Referências Bibliográficas

SUIT CSS. *Official Documentation*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SUIT CSS. *Naming Conventions*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

WILLIAMS, Alex. *CSS Architecture and Design Systems*. New York: Apress, 2021.

Exemplo Prático da Metodologia SUIT CSS

Introdução

A metodologia SUIT CSS tem como objetivo organizar o código CSS por meio de uma convenção de nomenclatura consistente e orientada a componentes. Cada elemento da interface é tratado como um componente independente, contendo descendentes, modificadores e estados claramente identificáveis.

Neste exemplo será desenvolvido um componente de **curso acadêmico** para um portal universitário. O componente exibirá informações sobre um curso disponível para matrícula e demonstrará a aplicação dos principais conceitos do SUIT CSS.

Desenvolvimento

Cenário

Uma universidade deseja disponibilizar um catálogo de cursos de extensão. Cada curso deve apresentar:

- Imagem do curso;
- Nome do curso;
- Descrição resumida;
- Carga horária;
- Botão de inscrição.

Além disso, alguns cursos podem receber destaque na página principal.

Estrutura HTML

```
<article class="CourseCard CourseCard--featured">
```

```
  
```

```
  <div class="CourseCard-content">
```

```
    <h2 class="CourseCard-title">
      Desenvolvimento Web Avançado
    </h2>
```

```
    <p class="CourseCard-description">
      Curso voltado ao desenvolvimento de aplicações
      web modernas utilizando arquiteturas escaláveis.
    </p>
```

```
    <p class="CourseCard-workload">
      Carga Horária: 80 horas
    </p>
```

```
    <button class="Button Button--primary">
      Inscrever-se
    </button>
```

```
  </div>
```

```
</article>
```

Observe a utilização de:

- `CourseCard` → componente principal;
 - `CourseCard-image`, `CourseCard-content`, `CourseCard-title`, `CourseCard-description` e `CourseCard-workload` → descendentes;
 - `CourseCard--featured` → modificador;
 - `Button` → componente independente reutilizável;
 - `Button--primary` → modificador do botão.
-

CSS do Componente

Componente Principal

```
.CourseCard {  
  display: flex;  
  gap: 20px;  
  padding: 20px;  
  border: 1px solid #cccccc;  
  border-radius: 8px;  
  background-color: #ffffff;  
}
```

Descendentes

```
.CourseCard-image {  
  width: 250px;  
  border-radius: 6px;  
}
```

```
.CourseCard-content {  
  flex: 1;  
}
```

```
.CourseCard-title {  
  font-size: 1.8rem;  
  margin-bottom: 10px;  
}
```

```
.CourseCard-description {
```

```
    line-height: 1.6;
    margin-bottom: 15px;
}
```

```
.CourseCard-workload {
    font-weight: bold;
}
```

Cada descendente é claramente associado ao componente principal.

Modificador do Componente

Alguns cursos precisam receber destaque na página inicial.

```
.CourseCard--featured {
    border: 2px solid #0055aa;
    background-color: #eef6ff;
}
```

Aplicação:

```
<article class="CourseCard CourseCard--featured">
```

O modificador altera apenas a aparência do componente sem modificar sua estrutura.

Componente de Botão

O botão é tratado como um componente independente.

```
.Button {
    padding: 12px 20px;
    border: none;
    cursor: pointer;
}
```

Modificador:

```
.Button--primary {
    background-color: #0055aa;
    color: white;
}
```

Utilização:

```
<button class="Button Button--primary">
  Inscrever-se
</button>
```

Essa abordagem permite reutilizar o componente em qualquer área da aplicação.

Estados

Suponha que um curso esteja com inscrições encerradas.

```
.is-disabled {
  opacity: 0.5;
  pointer-events: none;
}
```

Aplicação:

```
<button
  class="Button Button--primary is-disabled">
  Inscrições Encerradas
</button>
```

O estado pode ser aplicado dinamicamente por JavaScript sem alterar a estrutura do componente.

Utilitários

SUIT CSS também incentiva a utilização de classes utilitárias.

```
.u-textCenter {
  text-align: center;
}
```

```
.u-marginTopLarge {
  margin-top: 30px;
}
```

Aplicação:

```
<h1 class="u-textCenter">
  Cursos Disponíveis
</h1>
```

Os utilitários possuem responsabilidade única e podem ser utilizados em diversos componentes.

Resultado Arquitetural

A estrutura do exemplo pode ser visualizada da seguinte forma:

CourseCard

```
|
├─ CourseCard-image
├─ CourseCard-content
|   ├─ CourseCard-title
|   └─ CourseCard-description
└─ CourseCard-workload
|
└─ Button
    └─ Button--primary
```

Estados:

```
└─ is-disabled
```

Utilitários:

```
├─ u-textCenter
└─ u-marginTopLarge
```

Essa organização facilita a identificação da responsabilidade de cada classe dentro do projeto.

Conclusão

O exemplo demonstra como a metodologia SUIT CSS organiza a construção de interfaces por meio de componentes independentes e nomenclatura padronizada. A utilização de componentes em PascalCase, descendentes com hífen, modificadores com `--`, estados com `is-` e utilitários com `u-` produz um código mais legível e previsível.

Em projetos de médio e grande porte, essa padronização reduz ambiguidades, melhora a colaboração entre equipes e facilita a manutenção das folhas de estilo ao longo do ciclo de vida da aplicação.

Referências Bibliográficas

SUIT CSS. *Official Documentation*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SUIT CSS. *Naming Conventions*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

WILLIAMS, Alex. *CSS Architecture and Design Systems*. New York: Apress, 2021.

Exercício Prático – Aplicando a Metodologia SUIT CSS

Introdução

A metodologia SUIT CSS foi criada para promover a construção de interfaces modulares por meio de uma convenção padronizada de nomenclatura. Sua principal característica é a organização dos estilos em componentes independentes, descendentes, modificadores, estados e utilitários.

Neste exercício, o estudante deverá desenvolver componentes para um sistema de biblioteca digital universitária, aplicando corretamente as convenções de nomenclatura do SUIT CSS e explorando os conceitos de reutilização e modularização.

Desenvolvimento

Situação-Problema

Uma universidade está desenvolvendo uma biblioteca digital para disponibilizar livros, artigos científicos e materiais didáticos aos estudantes.

A página principal deverá apresentar cartões contendo informações dos livros disponíveis.

Cada cartão deverá possuir:

- Capa do livro;
- Título;
- Autor;
- Categoria;
- Descrição resumida;
- Botão para download.

O sistema também deverá suportar:

- Livros em destaque;
- Livros recém-adicionados;
- Livros indisponíveis para download.

Além disso, a interface deverá conter elementos de alinhamento e espaçamento reutilizáveis.

Questão 1

Identifique a categoria SUIT CSS adequada para cada item abaixo.

Elemento	Categoria
BookCard	?
BookCard-title	?
BookCard-author	?
BookCard--featured	?
is-disabled	?
u-textCenter	?
DownloadButton	?
DownloadButton--primary	?

Questão 2

Crie a estrutura HTML de um cartão de livro utilizando a nomenclatura SUIT CSS.

Dados:

- Livro: Arquitetura de Software Moderna;
- Autor: João Pereira;
- Categoria: Engenharia de Software;
- Situação: Livro em destaque.

Utilize:

- Componente principal;
 - Descendentes;
 - Modificador;
 - Componente de botão.
-

Questão 3

Desenvolva o CSS do componente principal BookCard.

O componente deve possuir:

- Layout flexível;
 - Espaçamento interno;
 - Borda;
 - Cantos arredondados;
 - Fundo branco.
-

Questão 4

Crie os descendentes necessários para representar:

- Imagem da capa;
- Conteúdo textual;
- Título;
- Autor;
- Categoria;
- Descrição.

Siga rigorosamente a sintaxe do SUIT CSS.

Questão 5

Crie os modificadores abaixo:

Livro em destaque

BookCard--featured

Livro recém-adicionado

BookCard--new

Defina estilos visuais apropriados para cada situação.

Questão 6

Crie um componente de botão reutilizável chamado:

DownloadButton

O componente deverá possuir os modificadores:

DownloadButton--primary

DownloadButton--secondary

Questão 7

Crie os estados:

is-loading

is-disabled

is-hidden

Explique em quais situações cada estado poderia ser utilizado na biblioteca digital.

Questão 8

Crie cinco classes utilitárias utilizando o prefixo u-.

As classes devem atender às seguintes necessidades:

- Centralização de texto;
 - Espaçamento superior;
 - Espaçamento inferior;
 - Ocultação de elementos;
 - Layout flexível.
-

Questão 9

Analise o código abaixo:

```
.book-card-destaque {  
  background-color: #eef6ff;  
  border: 2px solid #0055aa;  
}
```

```
.book-card-titulo {  
  font-size: 2rem;
```

```
}
```

```
.botao-download {  
    background-color: green;  
}
```

Explique quais problemas esse código apresenta em relação aos princípios do SUIT CSS.

Sua resposta deve abordar:

- Padronização;
 - Reutilização;
 - Legibilidade;
 - Escalabilidade.
-

Questão 10

Reescreva o código da questão anterior utilizando corretamente a nomenclatura do SUIT CSS.

Desafio Proposto

A biblioteca digital será ampliada e passará a oferecer:

- Revistas científicas;
- Trabalhos de conclusão de curso;
- Teses e dissertações;
- Vídeos educacionais.

Para cada novo módulo:

1. Defina o nome do componente principal.
2. Crie pelo menos quatro descendentes.
3. Crie dois modificadores.
4. Crie dois estados possíveis.
5. Crie exemplos de utilitários que poderiam ser reutilizados.

Apresente a estrutura completa seguindo rigorosamente as convenções do SUIT CSS.

Conclusão

Este exercício foi elaborado para que o estudante pratique todos os elementos fundamentais da metodologia SUIT CSS: componentes, descendentes, modificadores, estados e utilitários. A correta aplicação dessas convenções permite a construção de sistemas CSS mais organizados, previsíveis e fáceis de manter.

Ao concluir a atividade, o estudante deverá ser capaz de projetar arquiteturas CSS modulares e escaláveis, aplicando padrões amplamente utilizados em projetos corporativos e aplicações de grande porte.

Referências Bibliográficas

SUIT CSS. *Official Documentation*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SUIT CSS. *Naming Conventions*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

WILLIAMS, Alex. *CSS Architecture and Design Systems*. New York: Apress, 2021.

Resolução do Exercício Prático – Metodologia SUIT CSS

Introdução

Nesta resolução será aplicada a metodologia SUIT CSS na construção de componentes para uma biblioteca digital universitária. O objetivo é demonstrar a utilização correta de componentes, descendentes, modificadores, estados e utilitários, seguindo as convenções de nomenclatura estabelecidas pela metodologia.

A solução evidencia como a padronização do SUIT CSS contribui para a criação de interfaces escaláveis, legíveis e de fácil manutenção.

Desenvolvimento

Questão 1 – Classificação dos Elementos

Elemento	Categoria SUIT CSS
BookCard	Componente
BookCard-title	Descendente
BookCard-author	Descendente
BookCard--featured	Modificador
is-disabled	Estado
u-textCenter	Utilitário
DownloadButton	Componente
DownloadButton--primary	Modificador

Questão 2 – Estrutura HTML

```
<article class="BookCard BookCard--featured">

  

  <div class="BookCard-content">

    <h2 class="BookCard-title">
      Arquitetura de Software Moderna
    </h2>

    <p class="BookCard-author">
      João Pereira
    </p>

    <p class="BookCard-category">
```

```
        Engenharia de Software
    </p>

    <p class="BookCard-description">
        Estudo dos principais padrões arquiteturais
        utilizados no desenvolvimento de sistemas modernos.
    </p>

    <button
        class="DownloadButton DownloadButton--primary">
        Download
    </button>

</div>

</article>
```

Questão 3 – CSS do Componente Principal

```
.BookCard {
    display: flex;
    gap: 20px;
    padding: 20px;
    border: 1px solid #cccccc;
    border-radius: 8px;
    background-color: #ffffff;
}
```

O componente principal define a estrutura geral do cartão.

Questão 4 – Descendentes

```
.BookCard-image {
    width: 180px;
    border-radius: 6px;
}
```

```
.BookCard-content {
```

```
    flex: 1;
}

.BookCard-title {
    font-size: 1.8rem;
    margin-bottom: 10px;
}

.BookCard-author {
    font-weight: bold;
    margin-bottom: 8px;
}

.BookCard-category {
    color: #0055aa;
    margin-bottom: 10px;
}

.BookCard-description {
    line-height: 1.6;
}
```

Todos os elementos internos permanecem claramente associados ao componente BookCard.

Questão 5 – Modificadores

Livro em Destaque

```
.BookCard--featured {
    border: 2px solid #0055aa;
    background-color: #eef6ff;
}
```

Livro Recém-Adicionado

```
.BookCard--new {
    border: 2px solid #28a745;
    background-color: #f1fff4;
}
```

Aplicação:

```
<article class="BookCard BookCard--new">
```

Os modificadores alteram características visuais sem modificar a estrutura do componente.

Questão 6 – Componente de Botão

Componente Base

```
.DownloadButton {  
    padding: 12px 20px;  
    border: none;  
    cursor: pointer;  
    border-radius: 4px;  
}
```

Modificador Primário

```
.DownloadButton--primary {  
    background-color: #0055aa;  
    color: white;  
}
```

Modificador Secundário

```
.DownloadButton--secondary {  
    background-color: #666666;  
    color: white;  
}
```

Exemplo:

```
<button  
    class="DownloadButton DownloadButton--primary">  
    Download  
</button>
```

Questão 7 – Estados

Estado de Carregamento

```
.is-loading {  
    opacity: 0.7;  
}
```

Uso:

Quando o sistema está preparando o arquivo para download.

Estado Desabilitado

```
.is-disabled {  
    opacity: 0.5;  
    pointer-events: none;  
}
```

Uso:

Quando o arquivo está indisponível para acesso.

Estado Oculto

```
.is-hidden {  
    display: none;  
}
```

Uso:

Quando determinado conteúdo não deve ser exibido ao usuário.

Questão 8 – Utilitários

```
.u-textCenter {  
    text-align: center;  
}
```

```
.u-marginTopLarge {  
    margin-top: 30px;  
}
```

```
.u-marginBottomLarge {  
    margin-bottom: 30px;  
}
```

```
.u-hidden {  
    display: none;  
}
```

```
.u-flex {  
    display: flex;  
}
```

Exemplo:

```
<h1 class="u-textCenter">  
    Biblioteca Digital  
</h1>
```

Os utilitários podem ser reutilizados em qualquer componente da aplicação.

Questão 9 – Análise do Código

Código apresentado:

```
.book-card-destaque {  
    background-color: #eef6ff;  
    border: 2px solid #0055aa;  
}
```

```
.book-card-titulo {  
    font-size: 2rem;  
}
```

```
.botao-download {  
    background-color: green;  
}
```

Problemas de Padronização

O código não utiliza a convenção oficial do SUIT CSS baseada em PascalCase.

Problemas de Reutilização

Os nomes das classes são muito específicos e não favorecem a composição de componentes reutilizáveis.

Problemas de Legibilidade

Não é possível identificar claramente se uma classe representa componente, descendente, modificador ou estado.

Problemas de Escalabilidade

Em sistemas grandes, a ausência de convenções aumenta a probabilidade de conflitos e inconsistências.

Questão 10 – Código Corrigido

```
.BookCard--featured {  
    background-color: #eef6ff;  
    border: 2px solid #0055aa;  
}
```

```
.BookCard-title {  
    font-size: 2rem;  
}
```

```
.DownloadButton--primary {  
    background-color: green;  
    color: white;  
}
```

Agora cada elemento segue corretamente a nomenclatura da metodologia.

Desafio Proposto

Revistas Científicas

Componente

.JournalCard {}

Descendentes

.JournalCard-image {}

.JournalCard-title {}

.JournalCard-author {}

.JournalCard-summary {}

Modificadores

.JournalCard--featured {}

.JournalCard--new {}

Estados

.is-loading {}

.is-disabled {}

Trabalhos de Conclusão de Curso

Componente

.ThesisCard {}

Descendentes

.ThesisCard-title {}

.ThesisCard-student {}

.ThesisCard-advisor {}

.ThesisCard-summary {}

Modificadores

.ThesisCard--approved {}

.ThesisCard--featured {}

Estados

```
.is-hidden {}  
.is-loading {}
```

Teses e Dissertações

Componente

```
.DissertationCard {}
```

Descendentes

```
.DissertationCard-title {}  
.DissertationCard-author {}  
.DissertationCard-program {}  
.DissertationCard-abstract {}
```

Modificadores

```
.DissertationCard--awardWinner {}  
.DissertationCard--recent {}
```

Estados

```
.is-disabled {}  
.is-hidden {}
```

Vídeos Educacionais

Componente

```
.VideoCard {}
```

Descendentes

```
.VideoCard-thumbnail {}  
.VideoCard-title {}  
.VideoCard-instructor {}  
.VideoCard-duration {}
```

Modificadores

```
.VideoCard--featured {}  
.VideoCard--recommended {}
```

Estados

```
.is-playing {}  
.is-paused {}
```

Utilitários Reutilizáveis

```
.u-textCenter {}  
.u-flex {}  
.u-hidden {}  
.u-marginTopLarge {}  
.u-marginBottomLarge {}
```

Esses utilitários podem ser utilizados em todos os módulos apresentados.

Conclusão

A resolução demonstra a aplicação completa dos princípios do SUIT CSS. A utilização de componentes em PascalCase, descendentes associados ao componente principal, modificadores para variações visuais, estados para condições temporárias e utilitários para funcionalidades genéricas produz uma arquitetura CSS consistente e facilmente escalável.

Em projetos corporativos e aplicações de grande porte, essa padronização reduz ambiguidades, melhora a colaboração entre equipes e facilita significativamente a manutenção do código ao longo do tempo.

Referências Bibliográficas

SUIT CSS. *Official Documentation*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SUIT CSS. *Naming Conventions*. Disponível em: <https://suitcss.github.io>. Acesso em: 25 maio 2026.

SNOOK, Jonathan. *SMACSS: Scalable and Modular Architecture for CSS*. Disponível em: <https://smacss.com>. Acesso em: 25 maio 2026.

FRAIN, Ben. *Responsive Web Design with HTML5 and CSS*. 4. ed. Birmingham: Packt Publishing, 2022.

WILLIAMS, Alex. *CSS Architecture and Design Systems*. New York: Apress, 2021.

